GYM MEMBERSHIP MANAGEMENT SYSTEM

A PROJECT REPORT

submitted in partial fulfilment of the requirements of the course

BCSE302L Database Systems

by

Student name	Register number
Liz Alex	24BCE2978
Malika Balaji	24BAI0114
Nehaa Prabhu	24BAI0130

Under the guidance of

Prof. Lydia Jane G

School of Computer Science and Engineering (SCOPE)

Vellore Institute of Technology, Vellore 632014, Tamil Nadu, India

Fall Semester 2026-27 | October 2026

Source code: https://github.com/malikabalaji/gymhub

Abstract

Gyms and fitness centres manage tightly linked data about members, trainers, workout plans, group classes, attendance and payments, yet many small and mid-sized gyms still keep these records in registers and disconnected spreadsheets. The result is overbooked classes, missed or untracked dues, and no reliable view of member engagement or revenue. This project designs and implements the **Gym Membership Management System**, a database solution built on MongoDB 8, a document-oriented NoSQL database management system.

The design followed the full database life cycle: requirements analysis, an Enhanced Entity Relationship model with a USER generalization, a relational schema normalized to third normal form, and a document model that embeds bounded, always-read-together data such as workout exercises and class rosters while referencing unbounded data such as attendance and payments. The implemented database has seven collections protected by `$jsonSchema` validators, nineteen secondary indexes, one read-only view, two multi-document ACID transactions and two change-stream triggers. A Python application layer of 36 CRUD functions enforces ten referential integrity rules. A reproducible dataset of 1,016 documents was loaded.

Fifteen representative queries covering selection, joins, grouping, subqueries and set operations, eight analytical aggregation pipelines and a recursive `$graphLookup` over the member referral network were executed on a live MongoDB 8.3 replica set. Indexing reduced a member's attendance history query from 200,000 documents examined in 80.5 ms to 105 documents in 2.1 ms, and a partial index served the dues report at one tenth of the size of a full index. Transaction tests confirmed complete rollback on failure, and all six automated tests passed.

The project shows that a carefully designed document database can provide the integrity guarantees usually associated with relational systems while giving a gym the flexible schema, fast dashboard reads and built-in analytics it needs.

Keywords: MongoDB, NoSQL, document database, data modelling, aggregation pipeline, indexing, ACID transactions, gym management.

Table of Contents

Abstract ..... **i**

Table of Contents ..... **ii**

List of Figures ..... **v**

List of Tables ..... **vi**

Part B: Main Report **1**

Chapter 1: Introduction **1**

1.1 Background of the Study 1
1.2 Motivation 1
1.3 Problem Statement 1
1.4 Aim and Objectives 1
1.5 Scope of the Project 2
1.6 Limitations 2
1.7 Methodology Overview 2

Chapter 2: Literature Review **3**

2.1 Overview of the Application Domain and Its Data Needs 3
2.2 Review of Data Models 3
2.3 Review of Existing Systems and Related Work 4
2.4 Comparison of DBMS Platforms and Tools 4
2.5 Standards and Best Practices 4
2.6 Gap Identified 5
2.7 How This Project Addresses the Gap 5
2.8 Summary of the Chapter 5

Chapter 3: System Analysis and Requirements **6**

3.1 Existing System and Its Drawbacks 6
3.2 Proposed System and Its Advantages 6
3.3 Feasibility Study 6
3.3.1 Technical Feasibility 6
3.3.2 Economic Feasibility 6
3.3.3 Operational Feasibility 6
3.4 Users, Roles and Access Needs 7
3.5 Requirements Specification 7
3.5.1 Functional Requirements 7
3.5.2 Data Requirements 7
3.5.3 Non-Functional Requirements 8
3.5.4 Hardware Requirements 8
3.5.5 Software Requirements 8
3.6 Business Rules and Constraints 9

Chapter 4: Database Design **10**

4.1 Design Approach and Principles 10
4.2 System Architecture Diagram 10
4.3 Data Flow Diagram 11
4.4 Conceptual Design: ER / EER Diagram 12
4.4.1 Entities and Attributes 13
4.4.2 Relationships, Cardinality and Participation 13

4.4.3 Keys 13
4.4.4 Generalization and Specialization 14
4.5 Logical Design 14
4.6 Normalization 19
4.6.1 Functional Dependencies 19
4.6.3 Justified Denormalization 20
4.7 Physical Design 20
4.7.1 Data Types and Storage 20
4.7.2 Indexing Strategy 20
4.7.3 Partitioning 21
4.8 Integrity Constraints 21

Chapter 5: Implementation 22
5.1 Development Environment and Setup 22
5.2 DBMS, Tools and Libraries Used 22
5.5 Views and Indexes Created 22
5.7 Application Connectivity 23
5.8 Version Control and Project Repository 23
5.9 Challenges Faced and Solutions Adopted 23

Chapter 6: Query Design and Performance Tuning 24
6.1 Representative Queries 24
6.1.1 Selection 24
6.1.2 Joins 24
6.1.3 Grouping and Aggregation 25
6.1.4 Subqueries 26
6.1.5 Set Operations 27
6.1.6 Analytical Reports and Graph Query 28
6.2 Performance Tuning 28
6.2.1 Execution Plans with explain() 28
6.2.2 Benchmark at Scale 28
6.2.3 Other Tuning Measures 29

Chapter 7: Results and Discussion 30
7.1 Output Screenshots and Sample Reports 30
7.2 Evaluation Against Objectives 32
7.3 Comparison with the Existing System 32
7.4 Discussion of Findings 33
7.5 Limitations Observed 33

Chapter 8: Conclusion and Future Work 34
8.1 Summary of the Work 34
8.2 Key Contributions 34
8.3 Future Enhancements 34
8.4 Concluding Remarks 34

Part C: References and Appendices 35

References 35

Appendix A: Complete DDL Scripts 37

Appendix B: Sample Data and DML Scripts 41
B.1 Sample Documents 41
B.2 DML: CRUD Operations (gym_db/crud.py, complete) 44

Appendix C: Stored Procedures, Functions and Triggers (Full Code) 51

C.1 Transactions (gym_db/transactions.py)....51
C.2 Triggers (gym_db/triggers.py)....52
C.3 Aggregation Pipelines, Graph Queries and View (gym_db/aggregations.py)....54
C.4 Index Creation and Explain (gym_db/indexes.py)....57

Appendix D: Complete Query Set with Outputs....59
D.1 Outputs of the Query Set on MongoDB 8.3.7 (query_output.txt)....59
D.2 Output of the Index Benchmark (benchmark_output.txt)....62

Appendix E: Full-Size ER Diagram and Relational Schema....63

Appendix F: Application Screenshots....65

Appendix G: User Manual....70
G.1 Installation....70
G.2 Running the System....70
G.3 Everyday Tasks....70
G.4 Backup and Restore....70
G.5 Troubleshooting....71

List of Figures

Figure 4.1: Layered system architecture....10
Figure 4.2: Context diagram (level 0 DFD)....11
Figure 4.3: Level 1 data flow diagram....11
Figure 4.4: Enhanced Entity Relationship diagram (Chen notation, key attributes shown)....12
Figure 4.5: Relational schema in third normal form (12 relations)....14
Figure 4.6: MongoDB document model (orange arrows are ObjectId references)....15
Figure 6.1: Median query time with and without indexes at 200,000 documents....28
Figure 7.1: Collected revenue by month and membership tier (pipeline A1)....30
Figure 7.2: Attendance rate and fill rate by class (pipeline A2)....31
Figure E.1: Complete EER diagram with all attributes....63
Figure E.2: Full-size relational schema (3NF)....64
Figure F.1: Validator rejecting a member with an invalid tier....65
Figure F.2: Collections created and 1,016 sample documents loaded....65
Figure F.3: CRUD demonstration, create and read operations....65
Figure F.4: CRUD demonstration, update, delete and authentication....66
Figure F.5: Aggregation reports A1 to A5....66
Figure F.6: Aggregation reports A6 to A8 and the member_dashboard view....67
Figure F.7: Referral network with $graphLookup....67
Figure F.8: Indexes, explain() comparison and text search....68
Figure F.9: Transactions T1 and T2, commit and rollback....68
Figure F.10: Set operations Q13 to Q15....68
Figure F.11: Index benchmark at 200,000 documents....69
Figure F.12: Change-stream triggers TR1 and TR2....69

List of Tables

Table 1.1: Database life cycle followed in this project....2
Table 2.1: Comparison of data models for the gym domain....3
Table 2.2: Existing gym management systems....4
Table 2.3: DBMS platforms considered....4
Table 3.1: Cost estimate....6
Table 3.2: Users, roles and access rights....7
Table 3.3: Functional requirements....7
Table 3.4: Data requirements: entities, sources, volume and growth....7
Table 3.5: Non-functional requirements....8
Table 3.6: Hardware requirements....8
Table 3.7: Software requirements....8
Table 3.8: Business rules and how they are enforced....9
Table 4.1: Entities and their attributes....13
Table 4.2: Relationships, cardinality and participation....13
Table 4.3: Collection members....16
Table 4.4: Collection trainers....16
Table 4.5: Collection workout_plans....17
Table 4.6: Collection classes....17
Table 4.7: Collection class_sessions....17
Table 4.8: Collection attendance....18
Table 4.9: Collection payments....18
Table 4.10: Functional dependencies and normal form of each relation....19
Table 4.11: Denormalization decisions....20
Table 4.12: Indexes created....20
Table 4.13: Integrity constraints and their MongoDB implementation....21
Table 5.1: DBMS, tools and libraries....22
Table 5.2: Challenges and solutions....23
Table 6.1: Output of Q4 (first 4 rows)....25
Table 6.2: Output of Q7....26
Table 6.3: Output of Q9....26
Table 6.4: Report pipelines and graph queries....28
Table 6.5: Execution plans for the member attendance history query (sample data)....28
Table 6.6: Benchmark results....29
Table 7.1: Sample data loaded....30
Table 7.2: Class popularity report (A2)....31
Table 7.3: Members at risk (A3) and top dues (A4)....31
Table 7.4: Results of integrity, transaction and trigger tests....32
Table 7.5: Evaluation against the objectives of Section 1.4....32
Table 7.6: Existing manual system compared with the proposed system....32
Table C.1: Location of procedures, functions and triggers....51
Table G.1: Commands....70
Table G.2: Common problems....71

PART B: MAIN REPORT

CHAPTER 1
INTRODUCTION

1.1 Background of the Study

A modern gym offers tiered memberships, personal training, group classes and monthly billing. This data is interlinked: a member is coached by a trainer, follows that trainer's workout plan, enrols in classes, attends sessions and pays a tier-based fee. A DBMS stores such data once, enforces rules on it and answers questions efficiently [3], [4]. Relational systems [1] were long the default, but document databases such as MongoDB are now mainstream for nested, evolving data [5], [9], [10]. This project applies the course's design principles to this domain on MongoDB.

1.2 Motivation

Gyms near our campus use a paper register, a fee spreadsheet and a messaging group. Three problems recur:

- **Revenue leakage:** nobody sees which members owe money or for how long.
- **Poor class planning:** some classes are overbooked while others run half empty.
- **Weak retention:** members who stop attending are noticed only when they fail to renew.

Commercial software exists but is closed and subscription-priced, so we built an open system covering every stage of the database life cycle.

1.3 Problem Statement

Design and implement a database for a single gym that stores members, trainers, workout plans, classes, enrolment, attendance and payments; enforces the business rules that keep this data consistent; supports role-based access for administrators, trainers and members; produces revenue, popularity, dues and retention reports; and stays fast as history grows.

1.4 Aim and Objectives

Aim: build a reliable, well-designed and efficient gym database on MongoDB that meets real functional and non-functional requirements. The objectives (evaluated in Section 7.2) are:

- **O1.** Derive a document schema from a formal EER model.
- **O2.** Implement full CRUD for all core entities with referential integrity.
- **O3.** Enforce data quality in the database through validation, unique keys and constraints.
- **O4.** Provide analytics (revenue, popularity, attendance, dues, workload) through aggregation pipelines.
- **O5.** Provide authentication and role-based access for Administrator, Trainer and Member.
- **O6.** Ensure performance through evidence-based indexing.

- **O7.** Guarantee consistency of multi-document operations through transactions and triggers.
- **O8.** Deliver a reproducible, version-controlled implementation with data, tests and documentation.

1.5 Scope of the Project

In scope: single-gym member registration with Basic, Premium and VIP tiers; trainer profiles and personalised workout plans; class schedules, sessions and capacity-controlled enrolment; attendance; monthly billing with overdue flagging; analytical reports, a referral network, audit triggers and role-based login.

Out of scope: biometric or IoT check-in, online payment gateways, multi-branch support and a production web front end (planned as future work).

1.6 Limitations

- The sample data is synthetic, generated by a seeded script.
- One gym only; sharding was not implemented.
- The interface is command-line and mongosh; role-based screens are designed but not built.
- Referential integrity lives in the application layer because MongoDB has no foreign keys.
- Performance was measured on a single machine.

1.7 Methodology Overview

The project followed the database life cycle [3], [4] across three course reviews (Table 1.1).

Table 1.1: Database life cycle followed in this project

Phase	Outputs	Section
Requirements	User roles, 8 functional and 5 non-functional requirements, 12 business rules	Chapter 3
Conceptual design	EER diagram: 9 entities, 11 relationships, USER generalization	4.4
Logical design	3NF relational schema (12 relations), mapped to 7 collections	4.5, 4.6
Physical design	BSON types, 19 indexes, validators, replica set	4.7, 4.8
Implementation	`gym_db` package, mongosh DDL, data, view, transactions, triggers	Chapter 5
Testing	6 tests, 15 queries, explain plans, benchmark	Chapters 6, 7
Maintenance	Git repository, JSON export, user manual	5.8, Appendix G

CHAPTER 2
LITERATURE REVIEW

2.1 Overview of the Application Domain and Its Data Needs

Gym data has four characteristics that drive the database choice:

- **Strong relationships:** almost every record refers to a member, trainer or class.
- **Hierarchical data read together:** a plan with its exercises, a class with its schedule and roster.
- **Fast-growing history:** attendance and payments grow every week and dominate storage.
- **Analytics:** owners need revenue, attendance and dues aggregates, not single records.

Member contact and payment details are personal data under India's DPDP Act, 2023 [23], so passwords must be hashed and access restricted.

2.2 Review of Data Models

Table 2.1: Comparison of data models for the gym domain

Model	Strengths for a gym	Weaknesses for a gym
Relational [1], [3]	Foreign keys, checks, mature SQL	Nested data needs extra tables and joins; migrations for schema changes
Object-relational [24]	SQL plus arrays and JSON columns	Nesting is secondary; JSON weakly validated
Document [10]	Matches hierarchical data; flexible schema; rich aggregation; multi-document ACID [13]	No foreign keys; design needs discipline
Key-value	Very fast caching	No querying by attribute
Column-family [26]	Massive write throughput	Query patterns fixed in advance; no joins
Graph	Natural for referral networks	Poor fit for billing and reports
NewSQL [6], [27]	Distributed ACID SQL	Far more complexity than one gym needs

Reasons for the model chosen. The document model on MongoDB was selected because:

1. Exercises, schedules and rosters are stored with their owner and read in one operation.
2. Integrity is still achievable: `$jsonSchema` validators [11], unique indexes and multi-document transactions [13].
3. The aggregation framework provides joins, faceted reports and recursive graph traversal [12], [15] without a second database.
4. New fields, such as `referred_by` added after Review 1, need no migration.
5. MongoDB is the most widely used document database [22] and is covered in the syllabus.

The relational model was still used as the rigorous intermediate design (Section 4.5), and the document model is a justified denormalization of it (Section 4.6.3).

2.3 Review of Existing Systems and Related Work

Commercial platforms such as Mindbody [21a], Glofox [21b], Zen Planner [21c] and Gymdesk [21d] offer billing, booking, check-in and reports as subscriptions [21e].

Table 2.2: Existing gym management systems

System type	Provides	Limitation
Register and spreadsheets	Zero cost, familiar	No integrity, concurrency or analytics
Commercial SaaS	Billing, booking, check-in, apps	Recurring fee; closed data model; vendor holds the data
Academic gym projects	Relational CRUD screens	Rarely include analytics, index evidence or transactions

Cattell [5] and Stonebraker [8] frame the integrity versus flexibility trade-off, Brewer [7] explains the CAP balance and Pavlo and Aslett [6] describe NewSQL. Kleppmann [9] and MongoDB's modelling guide [10] give the rule used here: embed tree-like data loaded together, reference unbounded and many-to-many data.

2.4 Comparison of DBMS Platforms and Tools

Table 2.3: DBMS platforms considered

Criterion	MySQL 8	PostgreSQL 16	MongoDB 8	Cassandra 5
Data model	Relational	Object-relational	Document (BSON)	Wide column
Nested data	Separate tables	Arrays, JSONB	Native embedding	Collections
Joins	SQL	SQL	$lookup, $graphLookup	None
Transactions	ACID	ACID	Multi-document ACID	Per partition
Analytics	GROUP BY, windows	SQL plus extensions	Aggregation pipeline	Limited
Cost	Free	Free	Free Community Server	Free
Fit	Good, nesting awkward	Good	Best fit	Poor

Tools: **PyMongo** driver [29], **mongosh** and **Compass**, **bcrypt** [17], **pytest** with **mongomock**, and **Git/GitHub**.

2.5 Standards and Best Practices

- **Design:** Chen's ER model [2] and EER [3]; normalize to 3NF before denormalizing.
- **Queries:** an ISO SQL [20] equivalent is given for every query in Chapter 6.
- **Document modelling:** embed bounded data, reference unbounded data, extended reference pattern [10], [25].
- **Data quality:** `$jsonSchema` with strict validation and error action [11].
- **Security:** salted bcrypt hashes per OWASP [18] and NIST SP 800-63B [19]; hashes never returned.
- **Performance:** indexes verified with `explain("executionStats")` [16]; partial indexes [14].
- **Reliability:** replica set, majority write concern [13], mongodump backups.

2.6 Gap Identified

- Commercial systems do not let owners run custom analytics on their own data.
- Spreadsheets give no integrity, concurrency control or history.
- Academic projects stop at relational CRUD, without design justification, index measurements or transactions.
- Few student works show relational-grade integrity on a document database.

2.7 How This Project Addresses the Gap

- An open design that goes from EER to 3NF to a justified document model.
- Relational-grade integrity on MongoDB: validators, unique keys, referential actions, atomic updates, transactions.
- Analytics inside the database: 8 pipelines, a view and a recursive referral query.
- Measured performance with explain plans and a 200,000-document benchmark.
- A public repository with seeded data and tests so every result can be reproduced.

2.8 Summary of the Chapter

Gym data is related, hierarchical, fast-growing and analytical. After comparing data models, MongoDB's document model was chosen for its fit, validation, transactions, aggregation and zero cost. The project fills the gap of an open, measured and integrity-focused NoSQL design.

CHAPTER 3
SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Existing System and Its Drawbacks

The existing system is a front-desk register, a fee spreadsheet, paper class lists and trainers' notebooks. Its drawbacks: duplicate and inconsistent member records; nothing prevents overbooking, double payments or deleting members with dues; concurrent edits overwrite each other; no audit trail; reports take hours; and personal data is exposed.

3.2 Proposed System and Its Advantages

One MongoDB database of seven collections, used through a Python layer and mongosh scripts, giving:

- **Consistency:** validators, unique keys and ten referential rules.
- **Safe concurrency:** atomic updates and transactions prevent lost updates and overbooking.
- **Instant reports:** eight pipelines run in milliseconds.
- **Auditability and security:** a change-stream audit log, bcrypt hashes and role-based access.
- **Scalability:** indexes, replica set and a shard-ready design.

3.3 Feasibility Study

3.3.1 Technical Feasibility

MongoDB 8, Python and PyMongo are mature, free and run on macOS, Windows and Linux. The only special need, a replica set for transactions, is met by a single-node replica set on a laptop. The team had the required skills. Feasible.

3.3.2 Economic Feasibility

Table 3.1: Cost estimate

Item	Development	Yearly running
MongoDB, Python, PyMongo, mongosh, Compass	INR 0	INR 0
Hosting on the gym PC or Atlas free tier	INR 0	INR 0
Commercial gym SaaS (for comparison)	Not applicable	Monthly fee per location

No licence cost; feasible.

3.3.3 Operational Feasibility

Staff need three daily tasks (register members, record payments, mark attendance), each a single function. Trainers already plan workouts and take roll calls, so the system digitises existing habits. Feasible.

3.4 Users, Roles and Access Needs

Table 3.2: Users, roles and access rights

Role	Main tasks	Access
Administrator	Manage members, trainers, classes, sessions, payments; run reports	Full CRUD on all collections
Trainer	Create and update plans; take attendance; view rosters	Own members, classes and plans
Member	View profile, plan, schedule, attendance, dues; enrol	Own data through `member_dashboard`
System	Overdue sweep, audit log, reactivation	payments, members, audit_log

3.5 Requirements Specification

3.5.1 Functional Requirements

Table 3.3: Functional requirements

ID	Requirement	Implemented by
FR1	Admin CRUD on members, trainers, classes, payments	`crud.py`
FR2	Trainers manage personalised workout plans	`create_workout_plan`, `add_exercise`
FR3	Members view profile, plan, schedule, attendance, dues	`member_dashboard` view
FR4	Class scheduling with capacity limits and enrolment	`create_class`, `enroll_member`
FR5	Attendance per member per session	`mark_attendance`, T2
FR6	Payment tracking; overdue bills flagged	`record_payment`, `flag_overdue`
FR7	Revenue, popularity, attendance and workload reports	Pipelines A1 to A8
FR8	Email and password login with roles	`authenticate` (bcrypt)

3.5.2 Data Requirements

Table 3.4: Data requirements: entities, sources, volume and growth

Entity	Source	Sample	Expected growth
Member	Registration form	60	About 20 per month
Trainer	HR records	8	A few per year
Workout plan	Trainer	76	About 1.3 per member per year
Class	Administrator	8	Rarely changes
Class session	Schedule	64	About 16 per week
Attendance	Roll call	580	About 240 per week (12,000 per year)
Payment	Front desk	220	1 per member per month

Attendance and payments are unbounded, so they are kept in their own collections.

3.5.3 Non-Functional Requirements

Table 3.5: Non-functional requirements

ID	Category	Requirement and how it is met
NFR1	Performance	Dashboard queries under 100 ms; member history 2.1 ms on 200,000 rows (Section 6.2)
NFR2	Concurrency	No lost updates: atomic conditional updates and snapshot-isolated transactions
NFR3	Security	bcrypt hashes never returned; role-based access; audit trail
NFR4	Usability, availability	Simple commands; replica set failover when run with three nodes
NFR5	Consistency, backup	Validators, keys, transactions, journaling, mongodump and JSON export

3.5.4 Hardware Requirements

Table 3.6: Hardware requirements

Component	Minimum	Used
Processor	64-bit dual core	Apple M-series, x86-64 Linux
Memory	4 GB	8 GB and above
Storage	2 GB free	SSD

3.5.5 Software Requirements

Table 3.7: Software requirements

Software	Version	Purpose
MongoDB Community Server	8.0+ (tested 8.3.7, 8.3.11)	DBMS, replica set rs0
mongosh	2.x	DDL and query scripts
Python, PyMongo, bcrypt	3.10+, 4.x, 4.x	Application layer
pytest, mongomock	8.x, 4.x	Tests
Git	2.x	Version control

3.6 Business Rules and Constraints

Table 3.8: Business rules and how they are enforced

No.	Rule	Enforced by
BR1	One tier (Basic, Premium, VIP) and one status (active, expired, frozen) per member	Validator enum
BR2	Monthly fee: Basic 1,500, Premium 2,500, VIP 4,000 INR	`create_invoice`
BR3	Emails are unique	Unique indexes
BR4	At most one trainer per member, and it must exist	`create_member`
BR5	Plans come from the assigned trainer; one active plan per member	`create_workout_plan`
BR6	Only active members enrol; never above capacity; no duplicates	Atomic `enroll_member`
BR7	One attendance record per member per session	`uq_session_member`
BR8	Pending bills past due become overdue	`flag_overdue`
BR9	Paid bills cannot change	`record_payment`, `delete_payment`
BR10	No deleting members with dues or trainers with members or classes	`delete_member`, `delete_trainer`
BR11	Renewal records payment and new end date together or not at all	Transaction T1
BR12	Paying a bill reactivates an expired membership	Trigger TR2

CHAPTER 4
DATABASE DESIGN

4.1 Design Approach and Principles

The design went from a technology-independent EER model (4.4) to a 3NF relational schema (4.5, 4.6) and then, by documented denormalization, to a MongoDB document model (4.6.3). Principles:

- **Model for the queries:** each main screen needs one document read.
- **Embed bounded, reference unbounded** [9], [10].
- **Integrity first:** every rule in Section 3.6 is enforced by the database or one application function.
- **Evidence for performance:** every index serves a named query and is verified (Section 6.2).

4.2 System Architecture Diagram

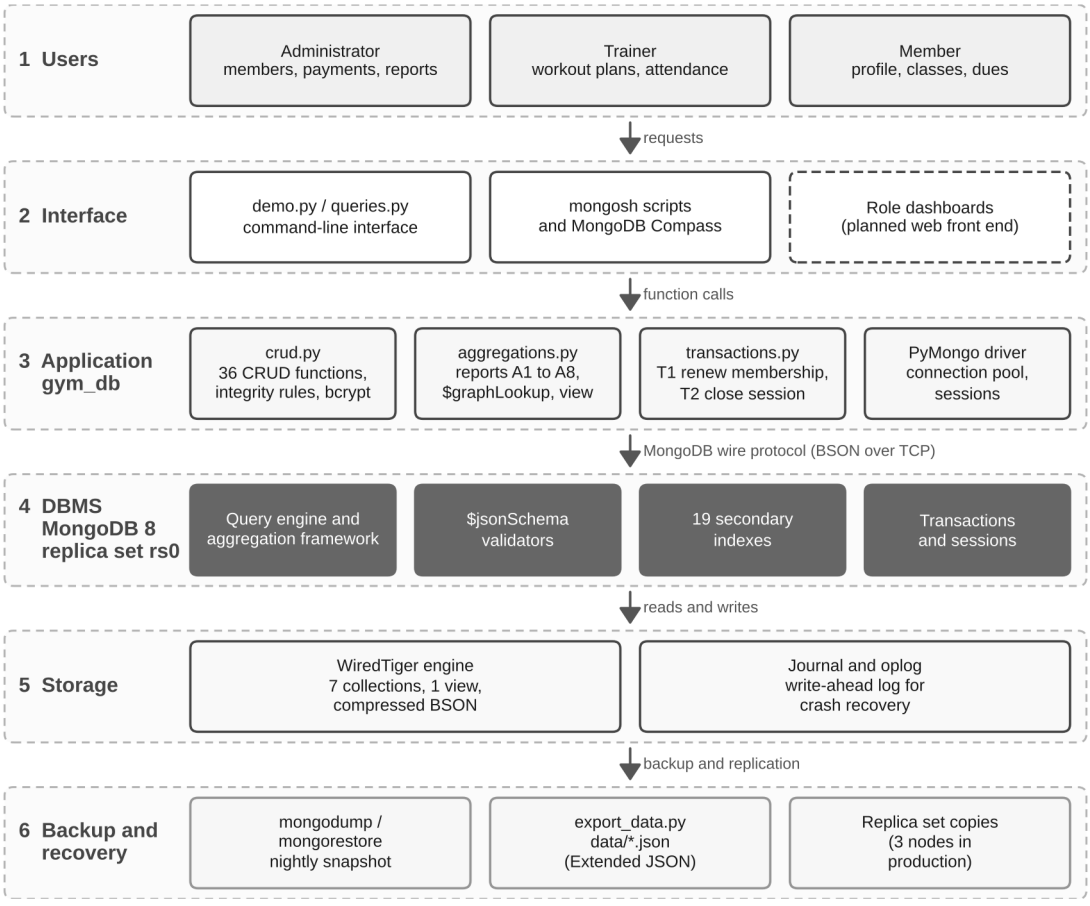


Figure 4.1: Layered system architecture

Users (administrator, trainer, member) work through the interface layer (`demo.py`, `queries.py`, mongosh, Compass). The application layer, the `gym_db` package, holds CRUD, reports, transactions and triggers and talks to MongoDB 8 (replica set rs0) through PyMongo.

WiredTiger stores compressed BSON with a journal and oplog, and mongodump plus JSON export provide backups.

4.3 Data Flow Diagram

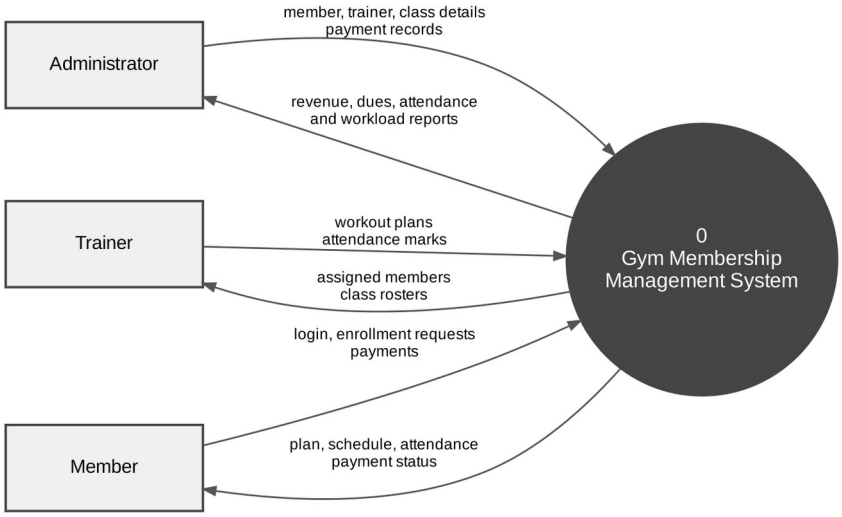


Figure 4.2: Context diagram (level 0 DFD)

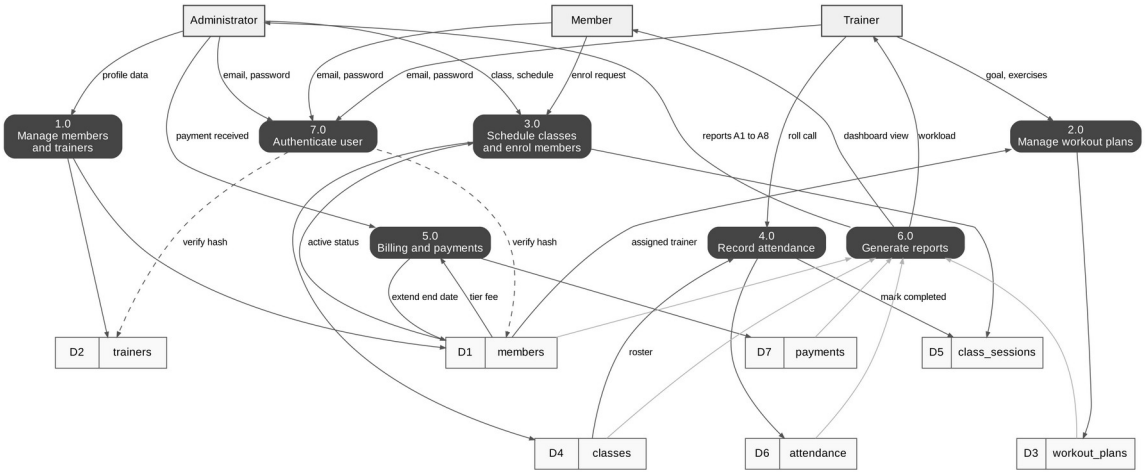


Figure 4.3: Level 1 data flow diagram

The level 1 DFD splits the system into seven processes (members and trainers, plans, classes and enrolment, attendance, billing, reporting, authentication). Data stores D1 to D7 are the seven collections.

4.4 Conceptual Design: ER / EER Diagram

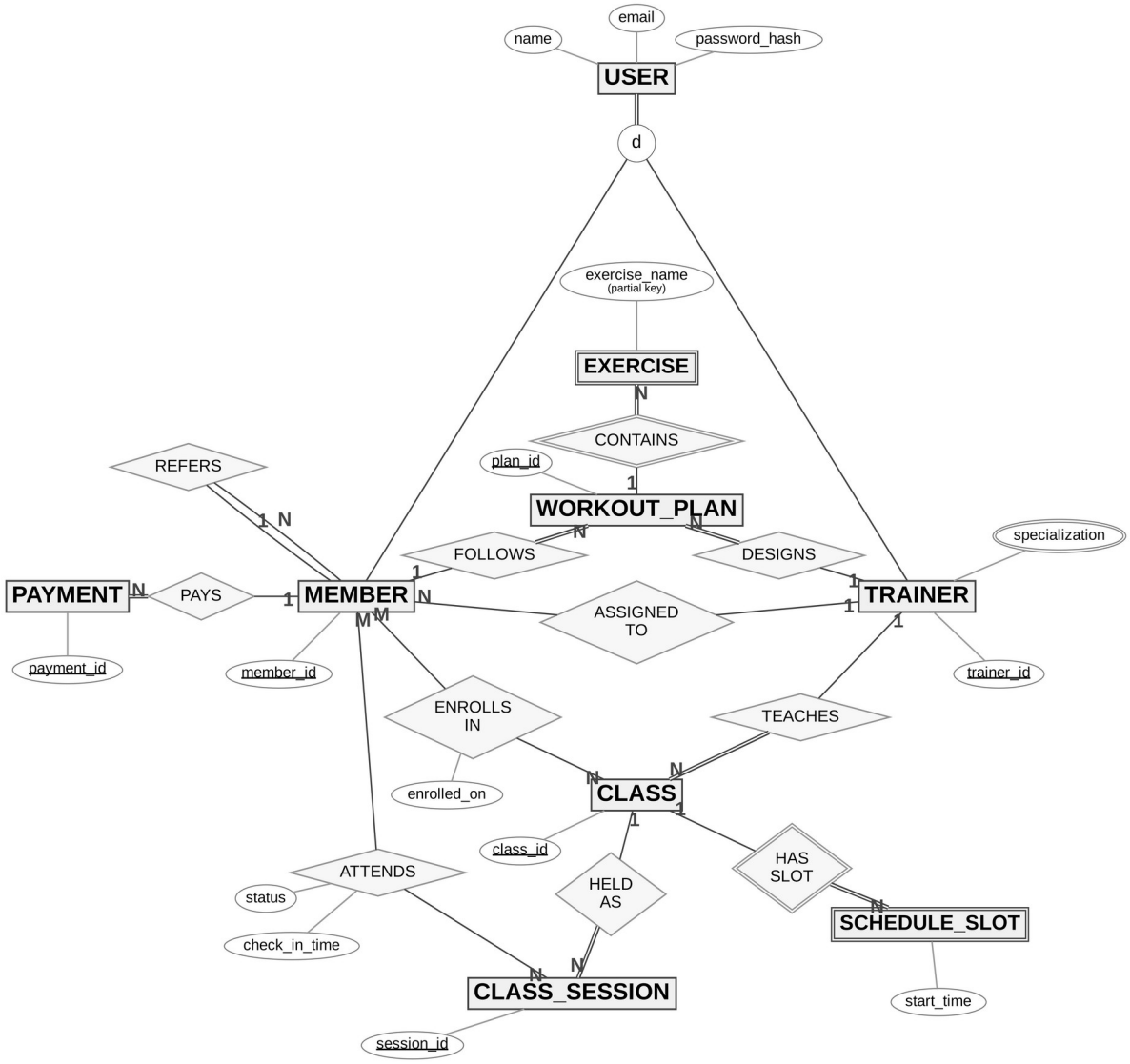


Figure 4.4: Enhanced Entity Relationship diagram (Chen notation, key attributes shown)

Chen notation: double rectangles are weak entities, double diamonds identifying relationships, double lines total participation and the circle d a disjoint specialization. The complete diagram is Figure E.1.

4.4.1 Entities and Attributes

Table 4.1: Entities and their attributes

Entity	Type	Attributes
USER	Superclass	name, email, phone, password_hash, role
MEMBER	Subclass	member_id (key), dob, gender, membership (type, status, start, end), address, emergency_contact
TRAINER	Subclass	trainer_id (key), certification, experience_years, specialization (multivalued), availability (multivalued)
WORKOUT_PLAN	Strong	plan_id (key), goal, duration_weeks, status, created_at, updated_at
EXERCISE	Weak	exercise_name (partial key), day, sets, reps, weight_kg, notes
CLASS	Strong	class_id (key), class_name, category, capacity
SCHEDULE_SLOT	Weak	day_of_week, start_time (partial key), end_time, room
CLASS_SESSION	Strong	session_id (key), date, start_time, end_time, status
PAYMENT	Strong	payment_id (key), amount, currency, mode, status, billing period, payment_date, due_date

4.4.2 Relationships, Cardinality and Participation

Table 4.2: Relationships, cardinality and participation

Relationship	Entities	Cardinality	Total participation
ASSIGNED_TO	TRAINER, MEMBER	1 : N	None
REFERS (recursive)	MEMBER, MEMBER	1 : N	None
FOLLOWS / DESIGNS	MEMBER / TRAINER, WORKOUT_PLAN	1 : N	WORKOUT_PLAN
CONTAINS (identifying)	WORKOUT_PLAN, EXERCISE	1 : N	EXERCISE
TEACHES	TRAINER, CLASS	1 : N	CLASS
ENROLLS_IN (enrolled_on)	MEMBER, CLASS	M : N	None
HAS_SLOT (identifying)	CLASS, SCHEDULE_SLOT	1 : N	SCHEDULE_SLOT
HELD_AS	CLASS, CLASS_SESSION	1 : N	CLASS_SESSION
ATTENDS (status, check_in_time)	MEMBER, CLASS_SESSION	M : N	None
PAYS	MEMBER, PAYMENT	1 : N	PAYMENT

4.4.3 Keys

- **Primary keys:** `_id` (ObjectId) on every document; readable business keys (MEM1001, TRN01, WP5001, CLS01, SES9001, PAY7001).
- **Candidate key:** `email` for MEMBER and TRAINER (unique index).
- **Partial keys:** EXERCISE by plan plus exercise_name; SCHEDULE_SLOT by class plus day and start time.

- **Relationship key:** ATTENDS by (session, member), unique index `uq_session_member`.
- **Foreign keys:** ObjectId references checked by the application (Section 4.8).

4.4.4 Generalization and Specialization

MEMBER and TRAINER specialise USER. The specialization is **disjoint and total**, mapped as one collection per subclass [3] because the two are always queried separately. The `role` field marks the subclass and login searches both collections.

4.5 Logical Design

The EER model was mapped to 12 relations in 3NF (Figure 4.5) with the standard algorithm [3]: weak entities and multivalued attributes became relations, the M:N relationships became ENROLLMENT and ATTENDANCE, and 1:N relationships became foreign keys.

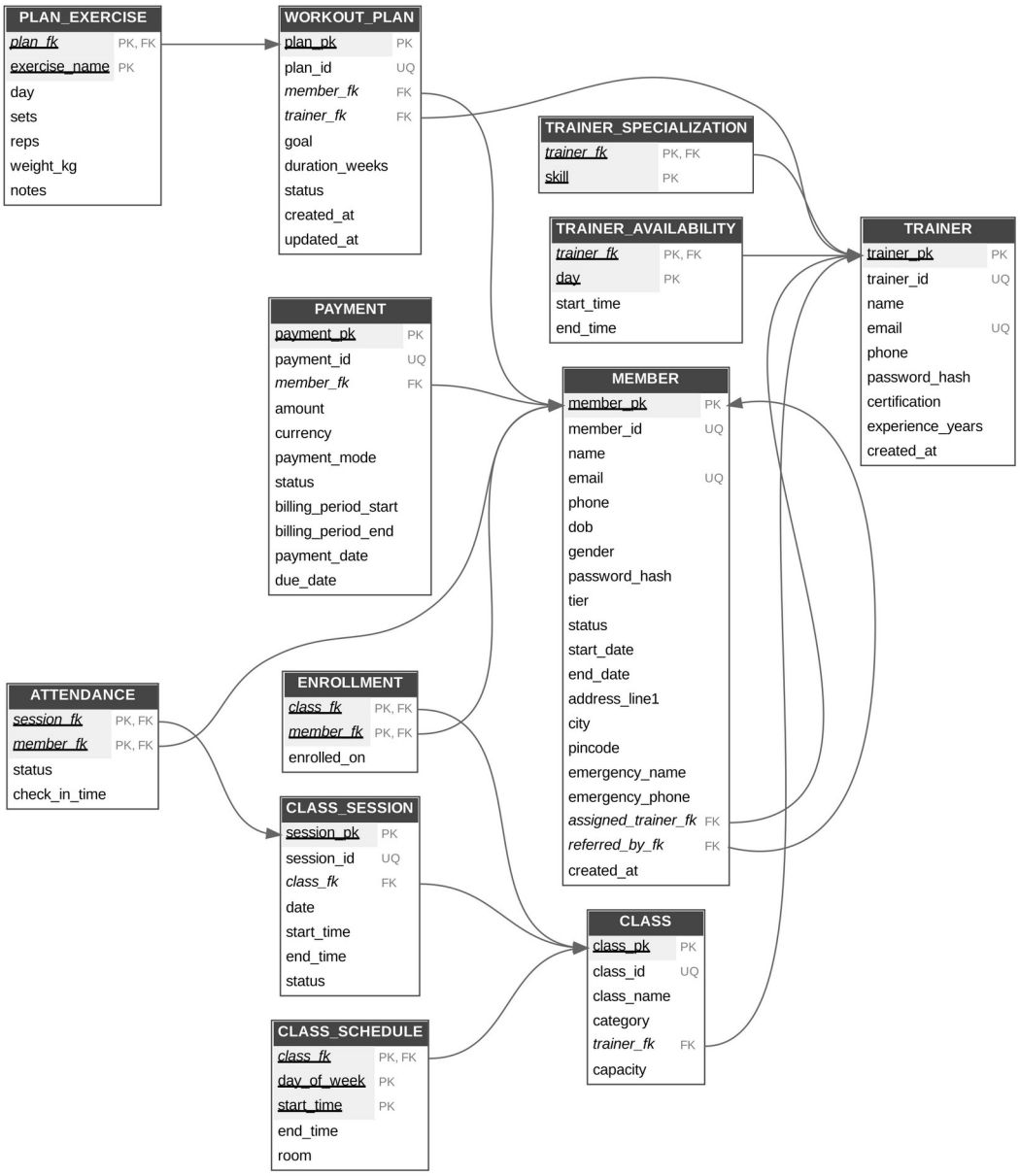


Figure 4.5: Relational schema in third normal form (12 relations)

Five relations were then embedded into their owners (specializations and availability into trainers, exercises into workout_plans, schedule and enrollment into classes), giving seven collections (Figure 4.6).

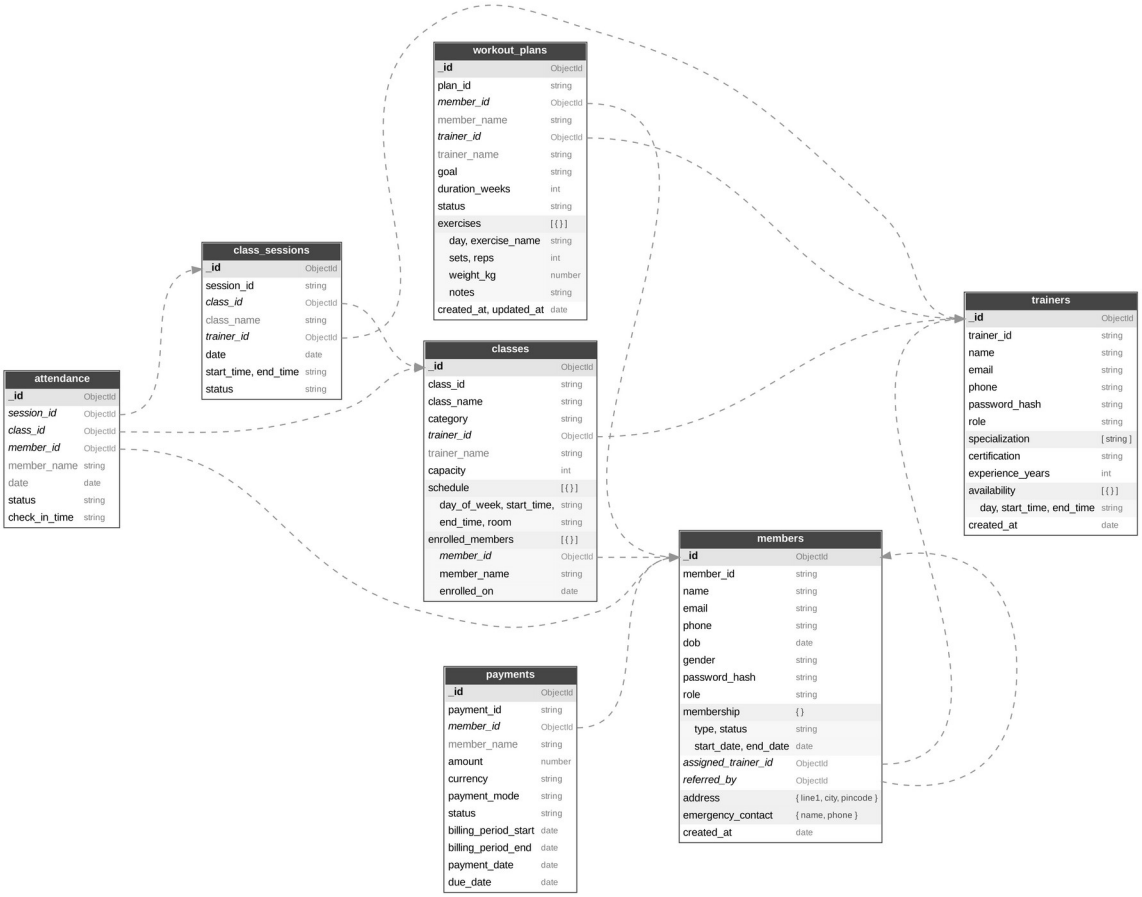


Figure 4.6: MongoDB document model (orange arrows are ObjectId references)

Tables 4.3 to 4.9 give the logical design of each collection. **A note on sizes:** BSON has no declared column widths. Fixed sizes apply only to ObjectId (12 bytes), int (4 bytes), double (8 bytes) and date (8 bytes). Strings are variable-length UTF-8, so the Size column shows the exact length where a validator pattern fixes it (for example `^MEM[0-9]{4}$` is 7 characters) and otherwise the expected maximum in the sample design, marked "var". Every limit the server actually enforces is listed in the Constraint column; "ref" marks an ObjectId reference checked by the application.

Table 4.3: Collection members

Field	Data type	Size	Constraint	Description
_id	ObjectId	12 B	Primary key	Internal identifier
member_id	string	7 chars	Required; unique; pattern `^MEM[0-9]{4}$`	Business key
name	string	var, 2 to 100	Required; minLength 2	Full name
email	string	var, up to 100	Required; unique; email pattern	Login name
phone	string	10 chars	Pattern `^[0-9]{10}$`	Mobile number
dob	date	8 B	BSON date	Date of birth
gender	string	up to 6	Enum Male, Female, Other	Gender
password_hash	string	60 chars	Required	bcrypt hash
role	string	6 chars	Required; enum member	Subclass marker
membership.type, .status	string	up to 7	Required; enums Basic, Premium, VIP and active, expired, frozen	Tier and status
membership.start_date, .end_date	date	8 B each	Required	Membership period
assigned_trainer_id	ObjectId or null	12 B	ref trainers._id	Assigned trainer
referred_by	ObjectId or null	12 B	ref members._id	Referring member
address, emergency_contact	object	var, about 180	Sub-fields line1, city, pincode; name, phone	Contact details
created_at	date	8 B	Default: insert time	Audit field

Table 4.4: Collection trainers

Field	Data type	Size	Constraint	Description
_id	ObjectId	12 B	Primary key	Internal identifier
trainer_id	string	5 chars	Required; unique; pattern `^TRN[0-9]{2}$`	Business key
name, email, phone, password_hash	string	as members	As in members; email unique	Shared USER attributes
role	string	7 chars	Required; enum trainer	Subclass marker
specialization	array of string	up to 10 items	String items; $addToSet avoids duplicates	Skills (multivalued)
certification	string	var, up to 30	None	Main certificate
experience_years	int	4 B	minimum 0	Years of experience
availability	array of object	up to 7 items	Each item: day, start_time, end_time	Weekly availability

Table 4.5: Collection workout_plans

Field	Data type	Size	Constraint	Description
_id	ObjectId	12 B	Primary key	Internal identifier
plan_id	string	6 chars	Required	Business key
member_id, trainer_id	ObjectId	12 B each	Required; ref members, trainers	Owner and author
member_name, trainer_name	string	var, up to 100	Copies	Denormalized for display
goal	string	var, up to 20	Required	Training goal
duration_weeks	int	4 B	minimum 1, maximum 52	Plan length
status	string	up to 9	Required; enum active, completed	One active plan per member
exercises	array of object	1 to about 10 items	minItems 1; sets, reps minimum 1	Embedded exercises
created_at, updated_at	date	8 B each	Default: time of change	Audit fields

Table 4.6: Collection classes

Field	Data type	Size	Constraint	Description
_id	ObjectId	12 B	Primary key	Internal identifier
class_id	string	5 chars	Required	Business key
class_name, category	string	var, up to 30	Required	Name and category
trainer_id, trainer_name	ObjectId, string	12 B, var	Required; ref trainers	Instructor
capacity	int	4 B	Required; minimum 1, maximum 50	Maximum enrolment
schedule	array of object	up to 14 items	day_of_week, start_time, end_time, room	Weekly slots
enrolled_members	array of object	up to capacity	member_id ref members; size never above capacity	Roster with member_name, enrolled_on

Table 4.7: Collection class_sessions

Field	Data type	Size	Constraint	Description
_id	ObjectId	12 B	Primary key	Internal identifier
session_id	string	7 chars	Required	Business key
class_id, trainer_id	ObjectId	12 B each	Required; ref classes, trainers	Parent class and instructor
class_name	string	var, up to 30	Copy	Denormalized
date	date	8 B	Required	Session date
start_time, end_time	string	5 chars	HH:MM	Time slot
status	string	up to 9	Required; enum scheduled, completed, cancelled	Lifecycle

Table 4.8: Collection attendance

Field	Data type	Size	Constraint	Description
_id	ObjectId	12 B	Primary key	Internal identifier
session_id, member_id	ObjectId	12 B each	Required; ref; unique together	Session and member
class_id	ObjectId	12 B	Required; ref classes	Copy for class reports
member_name	string	var, up to 100	Copy	Denormalized
date	date	8 B	Required	Copy of session date
status	string	up to 7	Required; enum present, absent, late	Outcome
check_in_time	string or null	5 chars	HH:MM	Arrival time

Table 4.9: Collection payments

Field	Data type	Size	Constraint	Description
_id	ObjectId	12 B	Primary key	Internal identifier
payment_id	string	7 chars	Required	Business key
member_id, member_name	ObjectId, string	12 B, var	Required; ref members	Payer
amount	double	8 B	Required; minimum 0	Bill amount in INR
currency	string	3 chars	Enum INR	Currency
payment_mode	string or null	up to 4	Enum cash, card, UPI, null	Null until paid
status	string	up to 7	Required; enum paid, pending, overdue	Lifecycle
billing_period_start, _end	date	8 B each	Dates	Period billed
payment_date	date or null	8 B	Set when paid	Date paid
due_date	date	8 B	Required	Payment deadline

4.6 Normalization

4.6.1 Functional Dependencies

Table 4.10: Functional dependencies and normal form of each relation

Relation	Functional dependencies	Normal form
MEMBER, TRAINER	pk → all; business id → pk; email → pk	BCNF
TRAINER_SPECIALIZATION	(trainer, skill) is the whole key	BCNF
TRAINER_AVAILABILITY	(trainer, day) → start_time, end_time	BCNF
WORKOUT_PLAN	plan_pk → member, trainer, goal, duration, status	BCNF
PLAN_EXERCISE	(plan, exercise_name) → day, sets, reps, weight_kg	BCNF
CLASS	class_pk → name, category, trainer, capacity	BCNF
CLASS_SCHEDULE	(class, day, start_time) → end_time, room	BCNF
ENROLLMENT	(class, member) → enrolled_on	BCNF
CLASS_SESSION	session_pk → class, date, times, status; (class, date, start_time) → session_pk	BCNF
ATTENDANCE	(session, member) → status, check_in_time	BCNF
PAYMENT	payment_pk → member, amount, currency, mode, status, dates; payment_id → payment_pk	BCNF

- **1NF:** multivalued attributes, repeating groups and rosters moved to their own relations; composites flattened.
- **2NF:** in every composite-key relation (PLAN_EXERCISE, ATTENDANCE, ENROLLMENT) non-key attributes depend on the whole key.
- **3NF:** no non-key attribute depends on another non-key attribute; for example trainer name is not stored in CLASS.
- **BCNF:** in every FD of Table 4.10 the left-hand side is a candidate key (a primary key, a business key such as payment_id, or email), so all twelve relations, PAYMENT included, are in BCNF and therefore also in 3NF.
- **PAYMENT check:** amount looks as if it depends on the member's tier, but tier is not an attribute of PAYMENT and the tier can change later while a past bill must not. Amount is therefore a fact about the bill itself (payment_pk → amount), not a transitive dependency, and no non-key determinant exists in PAYMENT.

4.6.3 Justified Denormalization

Table 4.11: Denormalization decisions

Decision	Reason	Consistency kept by
Embed exercises in workout_plans	Always read together; bounded	Atomic $push, $inc, $pull
Embed specializations and availability in trainers	Small lists read with profile	$addToSet
Embed schedule and roster in classes	Shown together; roster at most 50	Capacity check in update filter
Copy member, trainer and class names	Avoid a $lookup per row	`update_member` propagates renames
Store class_id and date in attendance	Reports without joining sessions	Copied at insert; sessions never move
Keep attendance and payments separate	Unbounded growth, 16 MB document limit	References plus indexes

4.7 Physical Design

4.7.1 Data Types and Storage

BSON types: ObjectId (12 bytes) for keys and references, string for codes and enums, int for counts, double for money and weights, date (8 bytes, UTC) for dates; times of day are HH:MM strings. WiredTiger compresses collections (Snappy) and indexes (prefix compression) and uses a journal for crash recovery. The largest document (a full class) is under 10 KB against the 16 MB limit [10].

4.7.2 Indexing Strategy

Table 4.12: Indexes created

Collection	Index	Keys (type)	Query served
members	uq_email, uq_member_id	email; member_id (unique)	Login, lookup
members	ix_trainer, ix_referred_by	assigned_trainer_id; referred_by	Workload; $graphLookup
members	ix_tier_status	membership.type, status (compound)	Member filters
trainers	uq_email, uq_trainer_id	email; trainer_id (unique)	Login, lookup
workout_plans	ix_member_status, ix_trainer	member_id, status; trainer_id	Active plan; trainer's plans
classes	ix_trainer	trainer_id	Workload, delete checks
classes	ix_enrolled_member	enrolled_members.member_id (multikey)	Member's classes
classes	tx_class_search	class_name, category (text)	Class search
class_sessions	ix_class_date	class_id, date	Timetable
attendance	ix_member_date	member_id, date desc	Member history
attendance	uq_session_member	session_id, member_id (unique)	One mark per session
attendance	ix_class_status	class_id, status	Popularity, absentees
payments	ix_member_status, ix_due_date	member_id, status; due_date	Dues; overdue sweep
payments	px_unpaid_dues	status, due_date (partial: unpaid)	Dues reports

Compound indexes follow the equality, sort, range rule, so member history needs no in-memory sort. The partial index stores only unpaid bills and stays small as paid history grows.

4.7.3 Partitioning

Not needed for one gym. For multiple branches, attendance and payments could be sharded on a hashed `member_id` (or `branch_id, member_id`). Replication (rs0) is used for availability.

4.8 Integrity Constraints

Table 4.13: Integrity constraints and their MongoDB implementation

Constraint	**Implementation**
Primary key, unique	`_id`; unique indexes on email, member_id, trainer_id, (session_id, member_id)
Not null, check	Validator `required`, `enum`, `pattern`, `minimum`/`maximum`, `minItems`
Default	Application defaults: status active, currency INR, created_at now
Foreign key	References checked before every insert and update (`_require`)
RESTRICT	Members with unpaid bills; trainers with members or classes
CASCADE	Deleting a member removes plans and roster entries; deleting a class removes future sessions
SET NULL	Deleting a member clears `referred_by` of members they referred
Cascade update	A rename updates every copied `member_name`
Cross-document rules	Atomic capacity check; transactions T1, T2; triggers TR1, TR2

CHAPTER 5
IMPLEMENTATION

5.1 Development Environment and Setup

Developed on macOS (Homebrew, MongoDB 8.3.11) and Linux (MongoDB 8.3.7), each as a single-node replica set for transactions and change streams. Full steps are in Appendix G.

```
brew install mongodb-community        # add replication: replSetName: rs0 to mongod.conf
brew services restart mongodb-community && mongosh --eval "rs.initiate()"
git clone https://github.com/malikabalaji/gymhub.git && cd gymhub
python3 -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt
python demo.py        # build, seed, demonstrate
```

5.2 DBMS, Tools and Libraries Used

Table 5.1: DBMS, tools and libraries

Component	Version	Role
MongoDB Community Server	8.3.7, 8.3.11	Server, replica set rs0
mongosh	2.14	`ddl.js`, `queries.js`
Python, PyMongo	3.12, 4.x	Application layer and driver
bcrypt	4.x	Password hashing
pytest, mongomock	8.x, 4.x	Offline tests
Git, GitHub	Latest	Version control

Repository layout: `gym_db/` (config, schema, indexes, seed, crud, aggregations, transactions, triggers), `demo.py`, `queries.py`, `benchmark.py`, `mongosh/` (DDL and queries), `data/` (JSON export), `tests/` and `docs/`.

5.5 Views and Indexes Created

View. `member_dashboard` joins each member's trainer name and payments, totals unpaid bills and hides the password hash (FR3):

```
MEMBER_DASHBOARD_PIPELINE = [
    {"$lookup": {"from": "trainers", "localField": "assigned_trainer_id", "foreignField": "_id",
        "as": "t"}},
    {"$lookup": {"from": "payments", "localField": "_id", "foreignField": "member_id", "as": "p"}},
    {"$project": {"_id": 0, "member_id": 1, "name": 1, "tier": "$membership.type",
                  "status": "$membership.status", "trainer": {"$first": "$t.name"},
                  "unpaid_amount": {"$sum": {"$map": {
                      "input": {"$filter": {"input": "$p", "as": "x", "cond": {"$ne": ["$$x.status",
                          "paid"]}}},
                      "as": "u", "in": "$$u.amount"}}}}},
]
```

Indexes. The 19 indexes of Table 4.12 are created by `indexes.py` and `mongosh/ddl.js` (Appendix A), for example:

```
db.attendance.createIndex({ member_id: 1, date: -1 }, { name: "ix_member_date" });
db.payments.createIndex({ status: 1, due_date: 1 },
  { name: "px_unpaid_dues", partialFilterExpression: { status: { $in: ["pending",
      "overdue"] } } });
```

Validators, transactions and triggers. Each collection has a `$jsonSchema` validator; a member with tier "Gold" is rejected with error 121 (Figure F.1). Transactions T1 (renew membership) and T2 (close a session), triggers TR1 (audit) and TR2 (reactivation) and all report pipelines are given in full in Appendix C.

5.7 Application Connectivity

PyMongo, the official driver, plays the role of JDBC or ODBC [29]. One pooled `MongoClient` is created from the `MONGO_URI` environment variable. `start_session()` with `with_transaction()` gives ACID transactions with majority write concern and automatic retry; `db.watch()` opens the change streams for triggers; documents map directly to Python dictionaries, so no ORM is needed; and `get_db(mock=True)` runs everything offline on mongomock.

5.8 Version Control and Project Repository

All code, data, outputs, reports and screenshots are on GitHub at **https://github.com/malikabalaji/gymhub**, with a README covering every command. Each review stage was committed separately.

5.9 Challenges Faced and Solutions Adopted

Table 5.2: Challenges and solutions

Challenge	Solution
No foreign keys in MongoDB	Ten referential rules in the application layer
Two members booking the last place at once	Capacity check inside the update filter (`$expr`, `$size`), one atomic operation
Transactions fail on a standalone server	Single-node replica set (`rs.initiate()`)
Homebrew rejected the untrusted tap	`brew trust mongodb/brew`
Keeping copied names consistent	`update_member` propagates renames, including inside arrays
Testing without a server	mongomock with a `$round` shim; server-only features verified live
Row order varied between runs	Tie-break sort keys
Index timings were 0 ms on 580 rows	`benchmark.py` with 200,000 documents

CHAPTER 6
QUERY DESIGN AND PERFORMANCE TUNING

6.1 Representative Queries

Fifteen queries cover selection, joins, grouping, subqueries and set operations. Each is shown in MongoDB syntax with its SQL equivalent and its output on the live MongoDB 8.3.7 replica set. Full outputs are in Appendix D.

6.1.1 Selection

Q1. Active VIP members in Chennai.

```
db.members.find({ "membership.type": "VIP", "membership.status": "active",
    "address.city": "Chennai" },
                { _id: 0, member_id: 1, name: 1,
                    "membership.end_date": 1 }).sort({ member_id: 1 })
-- SQL: SELECT member_id, name, end_date FROM members
--      WHERE tier='VIP' AND status='active' AND city='Chennai' ORDER BY member_id;
```

Output: MEM1014 Divya Das (ends 2026-12-31), MEM1043 Gautham Nair (2027-05-31), MEM1046 Nandini Menon (2027-02-28).

Q2. Trainers of Yoga or CrossFit with at least 6 years of experience.

```
db.trainers.find({ specialization: { $in: ["Yoga", "CrossFit"] },
    experience_years: { $gte: 6 } },
                { _id: 0, trainer_id: 1, name: 1,
                    experience_years: 1 }).sort({ experience_years: -1 })
-- SQL: SELECT DISTINCT t.trainer_id, t.name, t.experience_years FROM trainers t
--      JOIN trainer_specializations s ON s.trainer_id = t.trainer_id
--      WHERE s.skill IN ('Yoga','CrossFit') AND t.experience_years >= 6 ORDER BY 3 DESC;
```

Output: Deepa Krishnan (9), Vivek Nair (8), Kavitha Reddy (7), Priya Menon (6).

Q3. Unpaid bills past their due date, oldest first.

```
db.payments.find({ status: { $in: ["pending", "overdue"] },
    due_date: { $lt: ISODate("2026-09-24") } },
                { _id: 0, payment_id: 1, member_name: 1, amount: 1, due_date: 1 })
           .sort({ due_date: 1, payment_id: 1 }).limit(8)
-- SQL: SELECT ... FROM payments WHERE status IN ('pending','overdue')
--      AND due_date < DATE '2026-09-24' ORDER BY due_date, payment_id LIMIT 8;
```

Output: 17 bills match; the oldest is PAY7054 (Divya Menon, INR 1,500, due 2026-08-05), followed by sixteen bills due 2026-09-05.

6.1.2 Joins

Q4. Inner join: active VIP members with their trainer. `$unwind` drops non-matches, as an inner join does.

```
db.members.aggregate([
  { $match: { "membership.type": "VIP", "membership.status": "active" } },
  { $lookup: { from: "trainers", localField: "assigned_trainer_id", foreignField: "_id",
```

```
      as: "t" } },
  { $unwind: "$t" },
  { $project: { _id: 0, member_id: 1, name: 1, trainer: "$t.name" } },
      { $sort: { member_id: 1 } } ])
-- SQL: SELECT m.member_id, m.name, t.name FROM members m JOIN trainers t
--      ON t.trainer_pk = m.assigned_trainer_id WHERE m.tier='VIP' AND m.status='active';
```

Table 6.1: Output of Q4 (first 4 rows)

member_id	name	trainer
MEM1001	Harish Kumar	Priya Menon
MEM1012	Isha Das	Sanjay Pillai
MEM1014	Divya Das	Rohit Sharma
MEM1018	Aishwarya Das	Kavitha Reddy

Q5. Three-way join: attendance history of MEM1001 with class and session time.

```
db.members.aggregate([ { $match: { member_id: "MEM1001" } },
  { $lookup: { from: "attendance", localField: "_id", foreignField: "member_id", as: "a" } },
      { $unwind: "$a" },
  { $lookup: { from: "class_sessions", localField: "a.session_id", foreignField: "_id",
      as: "s" } }, { $unwind: "$s" },
  { $lookup: { from: "classes", localField: "a.class_id", foreignField: "_id", as: "c" } },
      { $unwind: "$c" },
  { $project: { _id: 0, date: "$s.date", class_name: "$c.class_name", status: "$a.status" } },
  { $sort: { date: -1 } } ])
-- SQL: SELECT s.date, c.class_name, a.status FROM members m
--      JOIN attendance a ON ... JOIN class_sessions s ON ... JOIN classes c ON ...
--      WHERE m.member_id = 'MEM1001' ORDER BY s.date DESC;
```

Output: five Evening Yoga sessions from 2026-09-09 to 2026-09-23, all present.

Q6. Left outer join: every trainer with the number of classes taught. `$lookup` without `$unwind` keeps trainers with no match.

```
db.trainers.aggregate([
  { $lookup: { from: "classes", localField: "_id", foreignField: "trainer_id", as: "c" } },
  { $project: { _id: 0, trainer_id: 1, name: 1, classes: { $size: "$c" } } },
  { $sort: { classes: -1, trainer_id: 1 } } ])
-- SQL: SELECT t.trainer_id, t.name, COUNT(c.class_id) FROM trainers t
--      LEFT OUTER JOIN classes c ON c.trainer_id = t.trainer_pk GROUP BY t.trainer_id, t.name;
```

Output: Deepa Krishnan teaches 2 classes, six trainers teach 1, and Priya Menon teaches 0 (kept by the outer join).

6.1.3 Grouping and Aggregation

Q7. Money collected by payment mode.

```
db.payments.aggregate([ { $match: { status: "paid" } },
  { $group: { _id: "$payment_mode", payments: { $sum: 1 }, total: { $sum: "$amount" },
      average: { $avg: "$amount" } } },
  { $sort: { total: -1 } } ])
-- SQL: SELECT payment_mode, COUNT(*), SUM(amount), AVG(amount) FROM payments
--      WHERE status='paid' GROUP BY payment_mode ORDER BY 3 DESC;
```

Table 6.2: Output of Q7

payment_mode	payments	total (INR)	average (INR)
UPI	101	237,500	2,351
card	37	97,000	2,622
cash	15	37,500	2,500

The total of INR 372,000 matches the revenue report A1 (Figure 7.1).

Q8. HAVING: cities with more than 10 members, by tier.

```
db.members.aggregate([
  { $group: { _id: "$address.city", members: { $sum: 1 },
      vip: { $sum: { $cond: [{ $eq: ["$membership.type", "VIP"] }, 1, 0] } } } },   // basic,
          premium alike
  { $match: { members: { $gt: 10 } } }, { $sort: { members: -1 } } ])               // HAVING
-- SQL: SELECT city, COUNT(*), SUM(CASE WHEN tier='VIP' THEN 1 ELSE 0 END)
--      FROM members GROUP BY city HAVING COUNT(*) > 10;
```

Output (members: Basic, Premium, VIP): Chennai 16 (6, 6, 4), Bengaluru 13 (6, 5, 2), Hosur 12 (4, 2, 6).

Q9. Attendance outcome by class category (join, then conditional sums).

```
db.attendance.aggregate([
  { $lookup: { from: "classes", localField: "class_id", foreignField: "_id", as: "c" } },
      { $unwind: "$c" },
  { $group: { _id: "$c.category", marked: { $sum: 1 },
      absent: { $sum: { $cond: [{ $eq: ["$status", "absent"] }, 1, 0] } } } },     // present,
          late alike
  { $sort: { marked: -1 } } ])
-- SQL: SELECT c.category, COUNT(*), SUM(CASE WHEN a.status='absent' THEN 1 ELSE 0 END)
--      FROM attendance a JOIN classes c ON c.class_pk = a.class_id GROUP BY c.category;
```

Table 6.3: Output of Q9

category	marked	present	late	absent
Yoga	160	103	8	49
Zumba	95	69	8	18
CrossFit	75	51	10	14
Strength	70	46	7	17
HIIT	68	46	8	14
Pilates	60	46	5	9
Cardio	52	36	4	12

6.1.4 Subqueries

Q10. Scalar subquery: members below the gym-wide attendance rate. `$facet` computes the average and per-member rates in one pass.

```
db.attendance.aggregate([
  { $addFields: { attended: { $cond: [{ $eq: ["$status", "absent"] }, 0, 100] } } },
  { $facet: { overall:    [ { $group: { _id: null, avg: { $avg: "$attended" } } } ],
              per_member: [ { $group: { _id: "$member_name",
```

```
                rate: { $avg: "$attended" } } } ] } },
  { $unwind: "$overall" }, { $unwind: "$per_member" },
  { $match: { $expr: { $lt: ["$per_member.rate", "$overall.avg"] } } } ])
-- SQL: SELECT member_id, AVG(...) AS rate FROM attendance GROUP BY member_id
--      HAVING rate < (SELECT AVG(...) FROM attendance);
```

Output: the gym average is 77.1%; the lowest rates are Isha Nair 20%, Keerthana Iyer 40% and Meera Varma 45.8%.

Q11. Correlated subquery: members with more than one unpaid bill. `$lookup` with `let` and `pipeline` runs an inner query per member.

```
db.members.aggregate([
  { $lookup: { from: "payments", let: { mid: "$_id" }, as: "unpaid",
      pipeline: [ { $match: { $expr: { $and: [ { $eq: ["$member_id", "$$mid"] },
                                               { $in: ["$status", ["pending",
                                          "overdue"]] } ] } } } ] } },
  { $match: { $expr: { $gt: [{ $size: "$unpaid" }, 1] } } },
  { $project: { _id: 0, member_id: 1, name: 1, amount_due: { $sum: "$unpaid.amount" } } } ])
-- SQL: SELECT m.member_id, m.name FROM members m WHERE (SELECT COUNT(*) FROM payments p
--      WHERE p.member_id = m.member_pk AND p.status IN ('pending','overdue')) > 1;
```

Output: 17 members, each with two unpaid bills, for example MEM1008 Isha Joshi (INR 3,000) and MEM1014 Divya Das (INR 8,000).

Q12. EXISTS: classes taught by trainers who coach more than 8 members.

```
db.classes.aggregate([
  { $lookup: { from: "members", localField: "trainer_id", foreignField: "assigned_trainer_id",
      as: "m" } },
  { $match: { $expr: { $gt: [{ $size: "$m" }, 8] } } },
  { $project: { _id: 0, class_name: 1, trainer_name: 1, trainer_members: { $size: "$m" } } } ])
-- SQL: SELECT class_name FROM classes c WHERE EXISTS (SELECT 1 FROM members m
--      WHERE m.assigned_trainer_id = c.trainer_id GROUP BY 1 HAVING COUNT(*) > 8);
```

Output: Morning Yoga and Pilates Core (Deepa Krishnan, 12 members), Spin Cycle (Farhan Ali, 10), Strength 101 (Sanjay Pillai, 10).

6.1.5 Set Operations

`$setUnion`, `$setIntersection` and `$setDifference` work on arrays, so each operand is a class roster produced by `$facet`.

```
db.classes.aggregate([
  { $facet: { a: [ { $match: { class_name: "Morning Yoga" } },
      { $project: { n: "$enrolled_members.member_name" } } ],
              b: [ { $match: { class_name: "Evening Yoga" } },
                  { $project: { n: "$enrolled_members.member_name" } } ] } },
  { $project: { result: { $setUnion: [ { $arrayElemAt: ["$a.n", 0] }, { $arrayElemAt: ["$b.n",
      0] } ] } } } ])
-- SQL: SELECT member_name FROM enrollments WHERE class_name='Morning Yoga'
--      UNION SELECT member_name FROM enrollments WHERE class_name='Evening Yoga';
```

- **Q13 UNION:** Morning Yoga (13) and Evening Yoga (19) give **27** distinct members, so 5 attend both.
- **Q14 INTERSECT** (`$setIntersection`): **5** members are in both CrossFit Burn and Strength 101.

- **Q15 EXCEPT** (`$setDifference`, SQL `EXCEPT`): **3** of 50 active members are in no class (Ashwin Krishnan, Rahul Sharma, Swathi Varma), natural targets for a promotion.

6.1.6 Analytical Reports and Graph Query

Table 6.4: Report pipelines and graph queries

ID	Report	Main operators
A1	Monthly revenue by tier	$match, $lookup, two $group, $dateToString
A2	Class popularity	$group, $cond, $addToSet, $lookup, $round
A3	Members at risk (below 60%)	$group, $addFields, $match
A4	Outstanding dues with priority	Date arithmetic, $floor, nested $cond
A5	Trainer workload	Three $lookup, $filter, $size
A6	Membership overview	$facet with four sub-pipelines, $bucket
A7	Most prescribed exercises	$unwind on embedded array, $avg
A8	Peak check-in hours	$substr, $group
G1, G2	Referral network, top referrers	Recursive $graphLookup

6.2 Performance Tuning

6.2.1 Execution Plans with explain()

Table 6.5: Execution plans for the member attendance history query (sample data)

Plan	Stage	Docs examined	Returned	In-memory sort
Collection scan	COLLSCAN	580	5	Yes
ix_member_date	IXSCAN	5	5	No

The index cuts documents examined from 580 to 5 and returns them already sorted.

6.2.2 Benchmark at Scale

`benchmark.py` measures three queries on 200,000 attendance and 60,000 payment documents (median of 5 runs, MongoDB 8.3.7).

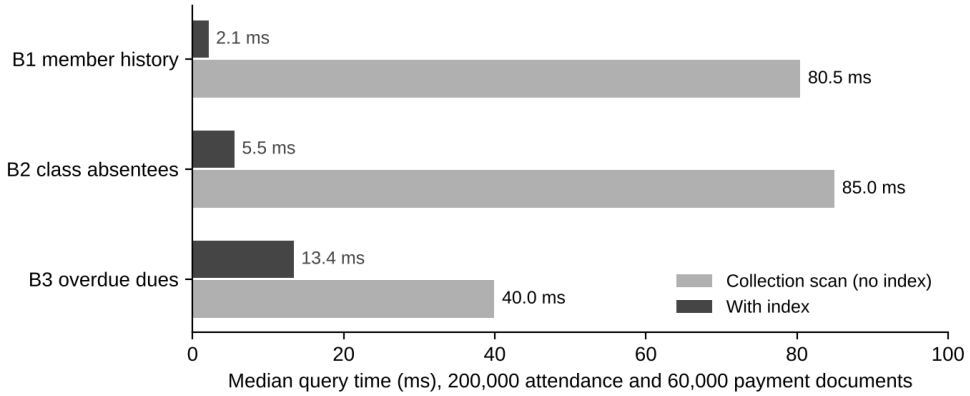


Figure 6.1: Median query time with and without indexes at 200,000 documents

Table 6.6: Benchmark results

Query	Plan	Docs examined	Median time
B1 member history	COLLSCAN	200,000	80.46 ms
B1 member history	ix_member_date	105	2.10 ms
B2 class absentees	COLLSCAN	200,000	84.97 ms
B2 class absentees	ix_class_status	998	5.53 ms
B3 overdue dues	COLLSCAN	60,000	39.95 ms
B3 overdue dues	px_unpaid_dues (partial)	2,965	13.38 ms
B3 overdue dues	Full index	2,965	12.78 ms

Member history became **38 times faster** and class absentees **15 times faster**. The partial index matched the full index in speed at **60 KB instead of 604 KB**.

6.2.3 Other Tuning Measures

- Projections return only needed fields and never the password hash.
- Every pipeline starts with `$match` so it can use an index.
- Copied names remove a `$lookup` from most list screens.
- Lists are paginated (sort, skip, limit).
- Enrolment, attendance and the overdue sweep are single atomic server-side updates.

CHAPTER 7
RESULTS AND DISCUSSION

7.1 Output Screenshots and Sample Reports

All results come from a live replica set: screenshots on macOS with MongoDB 8.3.11, query, benchmark and trigger outputs on 8.3.7. All screenshots are in Appendix F.

Table 7.1: Sample data loaded

Collection	Docs	Contents
trainers	8	Skills, certifications, availability
members	60	24 Basic, 22 Premium, 14 VIP; 50 active, 6 expired, 4 frozen
workout_plans	76	60 active, 16 completed; 4 exercises each
classes	8	Seven categories, 2 weekly slots each
class_sessions	64	7 September to 3 October 2026
attendance	580	Present, late or absent per enrolled member
payments	220	July to October 2026 bills
Total	**1,016**	All pass the validators

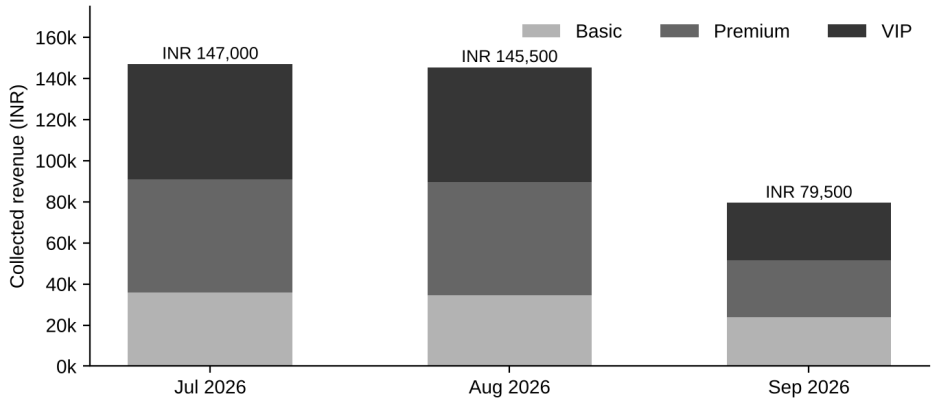


Figure 7.1: Collected revenue by month and membership tier (pipeline A1)

Revenue (A1): INR 147,000 in July, 145,500 in August and 79,500 in September; the drop comes from six memberships expiring on 31 August, four frozen and sixteen unpaid September bills. VIP members (14 of 60) produced 38% of July revenue.

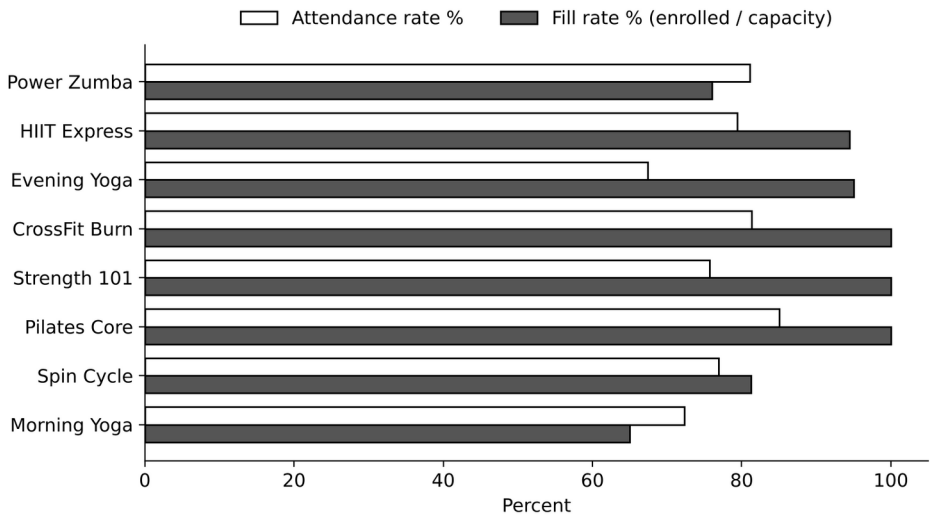


Figure 7.2: Attendance rate and fill rate by class (pipeline A2)

Table 7.2: Class popularity report (A2)

Class	Trainer	Enrolled / capacity	Avg turnout	Attendance %
Power Zumba	Meenakshi Iyer	19 / 25	15.4	81.1
HIIT Express	Rohit Sharma	17 / 18	13.5	79.4
Evening Yoga	Kavitha Reddy	19 / 20	12.8	67.4
CrossFit Burn	Vivek Nair	15 / 15	12.2	81.3
Strength 101	Sanjay Pillai	14 / 14	10.6	75.7
Pilates Core	Deepa Krishnan	12 / 12	10.2	85.0
Spin Cycle	Farhan Ali	13 / 16	10.0	76.9
Morning Yoga	Deepa Krishnan	13 / 20	9.4	72.3

Table 7.3: Members at risk (A3) and top dues (A4)

At risk: attendance below 60% (A3)	Highest priority dues (A4)
Isha Nair 20.0%, Keerthana Iyer 40.0%, Meera Varma 45.8%, Isha Joshi, Siddharth Varma, Gautham Joshi, Karthik Krishnan 50.0% (7 members)	Divya Menon INR 3,000, 50 days overdue; Divya Das, Aishwarya Das, Nandini Das, Nandini Menon INR 8,000 each, 19 days (all HIGH)

Other reports. Trainer workload (A5) ranges from Deepa Krishnan (12 members, 2 classes) to Meenakshi Iyer (2 members). The overview (A6) shows Chennai as the largest city (16) and 35 to 44 as the largest age band (22). Plank is the most prescribed exercise (A7, 23 plans) and 18:00 the busiest check-in hour (A8, 115). The referral query (G1, G2) shows that Lakshmi Bhat referred one member directly but five joined through her over three levels.

Table 7.4: Results of integrity, transaction and trigger tests

Test	Result on the live server
Member with tier "Gold"; duplicate email	Rejected: error 121; DuplicateKeyError
Member for missing trainer TRN77	Rejected: IntegrityError
Three enrolments in a class of capacity 2	Third blocked atomically
Rename a member	Name updated in all copies
Delete a member with unpaid bills	Blocked (RESTRICT)
T1 renewal with simulated failure	Rolled back: payments 221 to 221, end date unchanged
T1 renew MEM1010 for 3 months	Committed: INR 4,500; end date 31 Mar to 30 Jun 2027
T2 close SES9006 / with an invalid member	Committed 3 rows / rolled back, attendance 583 to 583
TR2 bill paid by expired MEM1004; TR1 audit	Membership reactivated; all changes logged
Automated tests	6 passed

Figures F.9 and F.13 show the transaction and trigger runs.

7.2 Evaluation Against Objectives

Table 7.5: Evaluation against the objectives of Section 1.4

ID	Evidence	Status
O1	EER, 3NF and document model (Figures 4.4 to 4.6)	Met
O2	36 CRUD functions, 10 referential rules (Table 7.4)	Met
O3	7 validators, unique indexes, rejection test (Figure F.1)	Met
O4	Q1 to Q15, A1 to A8, G1, G2	Met
O5	bcrypt login, roles, member_dashboard view	Met (web screens planned)
O6	19 indexes; 38 times faster; partial index 10 times smaller	Met
O7	T1, T2 commit and rollback; TR1, TR2	Met
O8	Public repository, seeded data, 6 tests, user manual	Met

7.3 Comparison with the Existing System

Table 7.6: Existing manual system compared with the proposed system

Aspect	Existing	Proposed
Integrity	None	Validators, keys, 10 referential rules
Overbooking	Possible	Prevented atomically
Dues	Manual, late	Overdue sweep and priority worklist
Reports	Hours of work	Milliseconds
Concurrency	Last edit wins	Atomic updates, transactions
Audit and security	None	Audit trigger, bcrypt, roles
Backup	Single copy	Journal, replica set, mongodump

7.4 Discussion of Findings

- **The document model fitted the domain:** the main screens each need one read, versus two to four joins in 3NF.
- **NoSQL integrity must be designed:** validators handle NOT NULL and CHECK; foreign keys need code; the capacity check in the update filter removes the race condition without locks.
- **Analytics are actionable:** Pilates Core is full with 85% attendance, while Morning Yoga is 65% full with the lowest turnout and could move to a new slot. Yoga has the most absences (31%). Two at-risk members (Isha Nair, Isha Joshi) also owe money, so retention and dues reports should be read together.
- **Revenue concentration:** UPI dominates (66% of paid bills) and VIP members matter most.
- **Indexes matter at scale:** no difference at 580 documents, 15 to 38 times faster at 200,000.

7.5 Limitations Observed

- Writes that bypass the application can skip referential rules (validators still apply).
- Copied names must be kept in sync.
- `$lookup` is slower than SQL joins on very large collections.
- Transactions and change streams need a replica set.
- Data is synthetic; concurrent real users were not tested.
- Roles are enforced in the application, not with MongoDB RBAC users.

CHAPTER 8
CONCLUSION AND FUTURE WORK

8.1 Summary of the Work

Following the full life cycle, the project captured 8 functional and 5 non-functional requirements and 12 business rules; built an EER model (9 entities, 11 relationships, USER generalization); normalized it to 12 relations in 3NF; and denormalized it into 7 MongoDB collections with 19 indexes and validators. The implementation has 36 CRUD functions, 8 pipelines, a referral query, a view, 2 transactions and 2 triggers over 1,016 documents. Fifteen queries, explain plans and a benchmark confirmed correctness and performance; all objectives were met.

8.2 Key Contributions

- A documented path from EER to 3NF to a justified document model.
- Relational-grade integrity on MongoDB.
- Fifteen queries mapping every SQL category, including set operations, to MongoDB.
- Measured evidence for each index, including a partial index.
- An open, reproducible repository.

8.3 Future Enhancements

- Web dashboard (Flask or React) on the existing `gym_db` layer.
- MongoDB RBAC users, field-level encryption and TLS.
- UPI or card payment gateway with transactional webhooks.
- SMS or email alerts for dues and at-risk members via change streams.
- Multi-branch support with sharding on a hashed member key.
- Churn prediction and class demand forecasting.

8.4 Concluding Remarks

Relational versus NoSQL is not a choice between integrity and flexibility. Starting from a rigorous relational design and denormalizing only where queries justify it, a document database gives a gym consistent, secure and fast data management with built-in analytics.

PART C: REFERENCES AND APPENDICES

References

[1] E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377 to 387, 1970.

[2] P. P. Chen, "The entity-relationship model: Toward a unified view of data," *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9 to 36, 1976.

[3] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA, USA: Pearson, 2016.

[4] A. Silberschatz, H. F. Korth and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[5] R. Cattell, "Scalable SQL and NoSQL data stores," *ACM SIGMOD Record*, vol. 39, no. 4, pp. 12 to 27, 2011.

[6] A. Pavlo and M. Aslett, "What's really new with NewSQL?," *ACM SIGMOD Record*, vol. 45, no. 2, pp. 45 to 55, 2016.

[7] E. Brewer, "CAP twelve years later: How the 'rules' have changed," *Computer*, vol. 45, no. 2, pp. 23 to 29, 2012.

[8] M. Stonebraker, "SQL databases v. NoSQL databases," *Communications of the ACM*, vol. 53, no. 4, pp. 10 to 11, 2010.

[9] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O'Reilly Media, 2017.

[10] MongoDB, Inc., "Data modeling," *MongoDB Manual*. [Online]. Available: https://www.mongodb.com/docs/manual/data-modeling/

[11] MongoDB, Inc., "Schema validation," *MongoDB Manual*. [Online]. Available: https://www.mongodb.com/docs/manual/core/schema-validation/

[12] MongoDB, Inc., "Aggregation operations," *MongoDB Manual*. [Online]. Available: https://www.mongodb.com/docs/manual/aggregation/

[13] MongoDB, Inc., "Transactions," *MongoDB Manual*. [Online]. Available: https://www.mongodb.com/docs/manual/core/transactions/

[14] MongoDB, Inc., "Partial indexes," *MongoDB Manual*. [Online]. Available: https://www.mongodb.com/docs/manual/core/index-partial/

[15] MongoDB, Inc., "$graphLookup (aggregation stage)," *MongoDB Manual*. [Online]. Available: https://www.mongodb.com/docs/manual/reference/operator/aggregation/graphLookup/

[16] MongoDB, Inc., "Explain results," *MongoDB Manual*. [Online]. Available: https://www.mongodb.com/docs/manual/reference/explain-results/

[17] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annual Technical Conference, FREENIX Track*, Monterey, CA, USA, 1999, pp. 81 to 91.

[18] OWASP Foundation, "Password storage cheat sheet," *OWASP Cheat Sheet Series*. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[19] P. A. Grassi et al., "Digital identity guidelines: Authentication and lifecycle management," National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST SP 800-63B, 2017.

[20] *Information technology: Database languages SQL, Part 1: Framework (SQL/Framework)*, ISO/IEC 9075-1:2023, International Organization for Standardization, 2023.

[21a] Mindbody, "Fitness studio and gym software." [Online]. Available: https://www.mindbodyonline.com

[21b] Glofox, "Gym management software." [Online]. Available: https://www.glofox.com

[21c] Zen Planner, "Gym management software." [Online]. Available: https://zenplanner.com

[21d] Gymdesk, "Gym management software." [Online]. Available: https://gymdesk.com

[21e] Wodify, "Best fitness studio management software for 2026: 7 platforms compared." [Online]. Available: https://www.wodify.com/blog/best-fitness-studio-software

[22] solid IT GmbH, "DB-Engines ranking." [Online]. Available: https://db-engines.com/en/ranking

[23] Government of India, *The Digital Personal Data Protection Act, 2023*, Act No. 22 of 2023, Ministry of Law and Justice, 2023.

[24] The PostgreSQL Global Development Group, "JSON types," *PostgreSQL Documentation*. [Online]. Available: https://www.postgresql.org/docs/current/datatype-json.html

[25] K. Banker, P. Bakkum, S. Verch, D. Garrett and T. Hawkins, *MongoDB in Action*, 2nd ed. Shelter Island, NY, USA: Manning, 2016.

[26] A. Lakshman and P. Malik, "Cassandra: A decentralized structured storage system," *ACM SIGOPS Operating Systems Review*, vol. 44, no. 2, pp. 35 to 40, 2010.

[27] R. Taft et al., "CockroachDB: The resilient geo-distributed SQL database," in *Proc. ACM SIGMOD International Conference on Management of Data*, 2020, pp. 1493 to 1509.

[28] MongoDB, Inc., "Change streams," *MongoDB Manual*. [Online]. Available: https://www.mongodb.com/docs/manual/changeStreams/

[29] MongoDB, Inc., "PyMongo documentation." [Online]. Available: https://pymongo.readthedocs.io/

Appendix A: Complete DDL Scripts

`mongosh/ddl.js` creates the seven collections with `$jsonSchema` validators, the nineteen indexes and the view. It is generated from the same definitions as `gym_db/schema.py`, so the two always agree.

```
// Generated by queries.py --export-js. Creates EMPTY collections (drops existing ones), then indexes.
// Load data afterwards with mongoimport (data/*.json) or use python demo.py, which does all of this.
//   mongosh "mongodb://localhost:27017/?directConnection=true" mongosh/ddl.js

// Data definition for gym_db: 7 collections with $jsonSchema validators, then 19 indexes.
db = db.getSiblingDB("gym_db");

db.getCollection("members").drop();
db.createCollection("members", {
  validator: { $jsonSchema: {
      bsonType: "object",
      required: ["member_id", "name", "email", "password_hash", "role", "membership"],
      properties: {
        member_id: { bsonType: "string", pattern: "^MEM[0-9]{4}$" },
        name: { bsonType: "string", minLength: 2 },
        email: { bsonType: "string", pattern: "^.+@.+\\..+$" },
        phone: { bsonType: "string", pattern: "^[0-9]{10}$" },
        gender: { enum: ["Male", "Female", "Other"] },
        role: { enum: ["member"] },
        membership: {
          bsonType: "object",
          required: ["type", "status", "start_date", "end_date"],
          properties: {
            type: { enum: ["Basic", "Premium", "VIP"] },
            status: { enum: ["active", "expired", "frozen"] },
            start_date: { bsonType: "date" },
            end_date: { bsonType: "date" }
          }
        },
        assigned_trainer_id: { bsonType: ["objectId", "null"] },
        referred_by: { bsonType: ["objectId", "null"] }
      }
    } },
  validationLevel: "strict",
  validationAction: "error"
});

db.getCollection("trainers").drop();
db.createCollection("trainers", {
  validator: { $jsonSchema: {
      bsonType: "object",
      required: ["trainer_id", "name", "email", "password_hash", "role"],
      properties: {
        trainer_id: { bsonType: "string", pattern: "^TRN[0-9]{2}$" },
        role: { enum: ["trainer"] },
        specialization: { bsonType: "array", items: { bsonType: "string" } },
        experience_years: { bsonType: ["int", "long", "double"], minimum: 0 },
        availability: {
          bsonType: "array",
          items: { bsonType: "object", required: ["day", "start_time", "end_time"] }
        }
      }
    } },
```

```
  validationLevel: "strict",
  validationAction: "error"
});

db.getCollection("workout_plans").drop();
db.createCollection("workout_plans", {
  validator: { $jsonSchema: {
      bsonType: "object",
      required: ["plan_id", "member_id", "trainer_id", "goal", "status", "exercises"],
      properties: {
        member_id: { bsonType: "objectId" },
        trainer_id: { bsonType: "objectId" },
        duration_weeks: { bsonType: ["int", "long"], minimum: 1, maximum: 52 },
        status: { enum: ["active", "completed"] },
        exercises: {
          bsonType: "array",
          minItems: 1,
          items: {
            bsonType: "object",
            required: ["day", "exercise_name", "sets", "reps"],
            properties: {
              sets: { bsonType: ["int", "long"], minimum: 1 },
              reps: { bsonType: ["int", "long"], minimum: 1 }
            }
          }
        }
      }
    } },
  validationLevel: "strict",
  validationAction: "error"
});

db.getCollection("classes").drop();
db.createCollection("classes", {
  validator: { $jsonSchema: {
      bsonType: "object",
      required: ["class_id", "class_name", "category", "trainer_id", "capacity"],
      properties: {
        trainer_id: { bsonType: "objectId" },
        capacity: { bsonType: ["int", "long"], minimum: 1, maximum: 50 },
        schedule: { bsonType: "array" },
        enrolled_members: { bsonType: "array" }
      }
    } },
  validationLevel: "strict",
  validationAction: "error"
});

db.getCollection("class_sessions").drop();
db.createCollection("class_sessions", {
  validator: { $jsonSchema: {
      bsonType: "object",
      required: ["session_id", "class_id", "date", "status"],
      properties: {
        class_id: { bsonType: "objectId" },
        date: { bsonType: "date" },
        status: { enum: ["scheduled", "completed", "cancelled"] }
      }
    } },
  validationLevel: "strict",
  validationAction: "error"
});
```

```
db.getCollection("attendance").drop();
db.createCollection("attendance", {
  validator: { $jsonSchema: {
      bsonType: "object",
      required: ["session_id", "class_id", "member_id", "date", "status"],
      properties: {
        session_id: { bsonType: "objectId" },
        member_id: { bsonType: "objectId" },
        status: { enum: ["present", "absent", "late"] }
      }
    } },
  validationLevel: "strict",
  validationAction: "error"
});

db.getCollection("payments").drop();
db.createCollection("payments", {
  validator: { $jsonSchema: {
      bsonType: "object",
      required: ["payment_id", "member_id", "amount", "status", "due_date"],
      properties: {
        member_id: { bsonType: "objectId" },
        amount: { bsonType: ["int", "long", "double"], minimum: 0 },
        currency: { enum: ["INR"] },
        payment_mode: { enum: ["cash", "card", "UPI", null] },
        status: { enum: ["paid", "pending", "overdue"] },
        due_date: { bsonType: "date" }
      }
    } },
  validationLevel: "strict",
  validationAction: "error"
});

db.members.createIndex({ email: 1 }, { unique: true, name: "uq_email" });
db.members.createIndex({ member_id: 1 }, { unique: true, name: "uq_member_id" });
db.members.createIndex({ assigned_trainer_id: 1 }, { name: "ix_trainer" });
db.members.createIndex({ "membership.type": 1, "membership.status": 1 }, { name: "ix_tier_status" });
db.members.createIndex({ referred_by: 1 }, { name: "ix_referred_by" });
db.trainers.createIndex({ email: 1 }, { unique: true, name: "uq_email" });
db.trainers.createIndex({ trainer_id: 1 }, { unique: true, name: "uq_trainer_id" });
db.workout_plans.createIndex({ member_id: 1, status: 1 }, { name: "ix_member_status" });
db.workout_plans.createIndex({ trainer_id: 1 }, { name: "ix_trainer" });
db.classes.createIndex({ trainer_id: 1 }, { name: "ix_trainer" });
db.classes.createIndex({ "enrolled_members.member_id": 1 }, { name: "ix_enrolled_member" });
db.classes.createIndex({ class_name: "text", category: "text" }, { name: "tx_class_search" });
db.class_sessions.createIndex({ class_id: 1, date: 1 }, { name: "ix_class_date" });
db.attendance.createIndex({ member_id: 1, date: -1 }, { name: "ix_member_date" });
db.attendance.createIndex({ session_id: 1, member_id: 1 }, { unique: true, name: "uq_session_member" });
db.attendance.createIndex({ class_id: 1, status: 1 }, { name: "ix_class_status" });
db.payments.createIndex({ member_id: 1, status: 1 }, { name: "ix_member_status" });
db.payments.createIndex({ due_date: 1 }, { name: "ix_due_date" });
db.payments.createIndex({ status: 1, due_date: 1 }, {
  name: "px_unpaid_dues",
  partialFilterExpression: { status: { "$in": ["pending", "overdue"] } }
});

// Read-only view used by the member dashboard (hides password_hash).
db.getCollection("member_dashboard").drop();
db.createView("member_dashboard", "members", [
  {
    "$lookup": { from: "trainers", localField: "assigned_trainer_id", foreignField: "_id", as: "t" }
  },
  { "$lookup": { from: "payments", localField: "_id", foreignField: "member_id", as: "p" } },
```

```
  {
    "$project": {
      _id: 0,
      member_id: 1,
      name: 1,
      tier: "$membership.type",
      status: "$membership.status",
      trainer: { "$first": "$t.name" },
      unpaid_amount: {
        "$sum": {
          "$map": {
            input: { "$filter": { input: "$p", as: "x", cond: { "$ne": ["$$x.status", "paid"] } } },
            as: "u",
            in: "$$u.amount"
          }
        }
      }
    }
  }
]);

print("gym_db created: " + db.getCollectionNames().sort().join(", "));
```

Appendix B: Sample Data and DML Scripts

B.1 Sample Documents

One document per collection (Extended JSON). The full dataset is in `data/*.json` (load with `mongoimport --jsonArray`); the generator is `gym_db/seed.py` (fixed seed).

members


```
{
  "_id": {
    "$oid": "6ae07293995dced28d294f9c"
  },
  "member_id": "MEM1001",
  "name": "Harish Kumar",
  "email": "harish.kumar1@example.com",
  "phone": "9952304698",
  "dob": {
    "$date": "1987-04-04T00:00:00Z"
  },
  "gender": "Male",
  "password_hash": "$2b$04$/zukbpdrHdQvDWSJSJ7LB.h77zHVCrIdcDUy8dQziLg22wwXkUJkm",
  "role": "member",
  "membership": {
    "type": "VIP",
    "status": "active",
    "start_date": {
      "$date": "2026-01-01T00:00:00Z"
    },
    "end_date": {
      "$date": "2026-12-31T00:00:00Z"
    }
  },
  "assigned_trainer_id": {
    "$oid": "6ae07293995dced28d294f94"
  },
  "referred_by": null,
  "address": {
    "line1": "193 Katpadi Main Road",
    "city": "Bengaluru",
    "pincode": "560034"
  },
  "emergency_contact": {
    "name": "Aishwarya Kumar",
    "phone": "9817771848"
  },
  "created_at": {
    "$date": "2025-12-19T00:00:00Z"
  }
}
```


workout_plans


```
{
  "_id": {
    "$oid": "6ae07293995dced28d294fd8"
  },
  "plan_id": "WP5001",
  "member_id": {
```

```
    "$oid": "6ae07293995dced28d294f9c"
  },
  "member_name": "Harish Kumar",
  "trainer_id": {
    "$oid": "6ae07293995dced28d294f94"
  },
  "trainer_name": "Priya Menon",
  "goal": "Flexibility",
  "duration_weeks": 10,
  "status": "active",
  "exercises": [
    {
      "day": "Mon",
      "exercise_name": "Plank",
      "sets": 4,
      "reps": 8,
      "weight_kg": 0,
      "notes": ""
    },
    {
      "day": "Tue",
      "exercise_name": "Sun Salutation",
      "sets": 4,
      "reps": 10,
      "weight_kg": 0,
      "notes": ""
    },
    {
      "day": "Wed",
      "exercise_name": "Cat-Cow Stretch",
      "sets": 3,
      "reps": 10,
      "weight_kg": 0,
      "notes": ""
    },
    {
      "day": "Fri",
      "exercise_name": "Hip Opener Flow",
      "sets": 5,
      "reps": 8,
      "weight_kg": 0,
      "notes": ""
    }
  ],
  "created_at": {
    "$date": "2026-01-01T00:00:00Z"
  },
  "updated_at": {
    "$date": "2026-01-01T00:00:00Z"
  }
}
```


classes (roster shortened)


```
{
  "_id": {
    "$oid": "6ae07293995dced28d295029"
  },
  "class_id": "CLS06",
  "class_name": "Pilates Core",
  "category": "Pilates",
  "trainer_id": {
    "$oid": "6ae07293995dced28d294f96"
```

```
  },
  "trainer_name": "Deepa Krishnan",
  "capacity": 12,
  "schedule": [
    {
      "day_of_week": "Tue",
      "start_time": "10:00",
      "end_time": "11:00",
      "room": "A2"
    },
    {
      "day_of_week": "Thu",
      "start_time": "10:00",
      "end_time": "11:00",
      "room": "A2"
    }
  ],
  "enrolled_members": [
    {
      "member_id": {
        "$oid": "6ae07293995dced28d294fa4"
      },
      "member_name": "Gautham Bhat",
      "enrolled_on": {
        "$date": "2026-07-11T00:00:00Z"
      }
    },
    {
      "member_id": {
        "$oid": "6ae07293995dced28d294fa3"
      },
      "member_name": "Isha Joshi",
      "enrolled_on": {
        "$date": "2026-07-12T00:00:00Z"
      }
    },
    {
      "...": "10 more"
    }
  ]
}
```


attendance


```
{
  "_id": {
    "$oid": "6ae07293995dced28d29506c"
  },
  "session_id": {
    "$oid": "6ae07293995dced28d29502c"
  },
  "class_id": {
    "$oid": "6ae07293995dced28d295024"
  },
  "member_id": {
    "$oid": "6ae07293995dced28d294fa8"
  },
  "member_name": "Manoj Shetty",
  "date": {
    "$date": "2026-09-07T00:00:00Z"
  },
  "status": "present",
  "check_in_time": "06:50"
```

```
}
```

payments

```
{
  "_id": {
    "$oid": "6ae07293995dced28d2952b0"
  },
  "payment_id": "PAY7001",
  "member_id": {
    "$oid": "6ae07293995dced28d294f9c"
  },
  "member_name": "Harish Kumar",
  "amount": 4000,
  "currency": "INR",
  "payment_mode": "card",
  "status": "paid",
  "billing_period_start": {
    "$date": "2026-07-01T00:00:00Z"
  },
  "billing_period_end": {
    "$date": "2026-07-31T00:00:00Z"
  },
  "payment_date": {
    "$date": "2026-07-08T00:00:00Z"
  },
  "due_date": {
    "$date": "2026-07-05T00:00:00Z"
  }
}
```

B.2 DML: CRUD Operations (gym_db/crud.py, complete)

All 36 insert, read, update and delete functions with the ten referential integrity rules.

```
"""Step 4: CRUD operations for every collection (FR1 to FR6, FR8).

MongoDB has no foreign keys, so referential integrity is enforced here (NFR5):
  * references are checked before insert/update,
  * denormalized names are propagated when the source changes,
  * deletes either cascade or are blocked when dependent data exists.
"""
from datetime import datetime

import bcrypt
from bson import ObjectId
from pymongo import ReturnDocument

from .config import REFERENCE_DATE, TIER_FEES

SAFE = {"password_hash": 0}  # never return password hashes to callers


class IntegrityError(Exception):
    pass


def _next_code(coll, field, prefix, width):
    last = coll.find_one({field: {"$regex": f"^{prefix}"}}, sort=[(field, -1)], projection={field: 1})
    n = int(last[field][len(prefix):]) + 1 if last else 1
    return f"{prefix}{n:0{width}d}"
```

```
def _require(db, coll, _id, label):
    doc = db[coll].find_one({"_id": _id})
    if not doc:
        raise IntegrityError(f"{label} {_id} does not exist")
    return doc


def _member_oid(db, member_id):
    doc = db.members.find_one({"member_id": member_id}, {"_id": 1})
    if not doc:
        raise IntegrityError(f"Member {member_id} not found")
    return doc["_id"]


# ---------------------------------------------------------------- MEMBERS
def create_member(db, name, email, phone, dob, gender, password, tier="Basic",
                  trainer_id=None, referred_by=None, city="Vellore", pincode="632014"):
    trainer_oid = None
    if trainer_id:
        trainer_oid = db.trainers.find_one({"trainer_id": trainer_id}, {"_id": 1})
        if not trainer_oid:
            raise IntegrityError(f"Trainer {trainer_id} not found")
        trainer_oid = trainer_oid["_id"]
    ref_oid = _member_oid(db, referred_by) if referred_by else None
    start = REFERENCE_DATE.replace(day=1)
    doc = {
        "member_id": _next_code(db.members, "member_id", "MEM", 4),
        "name": name, "email": email.lower(), "phone": phone, "dob": dob, "gender": gender,
        "password_hash": bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode(),
        "role": "member",
        "membership": {"type": tier, "status": "active", "start_date": start,
                       "end_date": datetime(start.year + 1, start.month, 1)},
        "assigned_trainer_id": trainer_oid, "referred_by": ref_oid,
        "address": {"line1": "", "city": city, "pincode": pincode},
        "emergency_contact": {"name": "", "phone": ""},
        "created_at": REFERENCE_DATE,
    }
    db.members.insert_one(doc)
    return doc["member_id"]


def get_member(db, member_id):
    return db.members.find_one({"member_id": member_id}, SAFE)


def list_members(db, tier=None, status=None, city=None, page=1, page_size=10):
    q = {}
    if tier:
        q["membership.type"] = tier
    if status:
        q["membership.status"] = status
    if city:
        q["address.city"] = city
    cur = (db.members.find(q, {"member_id": 1, "name": 1, "membership": 1, "address.city": 1, "_id": 0})
           .sort("member_id", 1).skip((page - 1) * page_size).limit(page_size))
    return list(cur), db.members.count_documents(q)


def update_member(db, member_id, **fields):
    """Update profile fields. A name change is propagated to every denormalized copy."""
    allowed = {"name", "phone", "email", "address", "emergency_contact"}
    bad = set(fields) - allowed
```

```
    if bad:
        raise ValueError(f"Cannot update {bad} here")
    doc = db.members.find_one_and_update({"member_id": member_id}, {"$set": fields},
                                         projection=SAFE, return_document=ReturnDocument.AFTER)
    if doc and "name" in fields:
        oid, name = doc["_id"], fields["name"]
        for coll in ("workout_plans", "attendance", "payments"):
            db[coll].update_many({"member_id": oid}, {"$set": {"member_name": name}})
        db.classes.update_many({"enrolled_members.member_id": oid},
                               {"$set": {"enrolled_members.$.member_name": name}})  # positional $
    return doc


def change_membership(db, member_id, tier=None, status=None):
    s = {}
    if tier:
        if tier not in TIER_FEES:
            raise ValueError("tier must be Basic, Premium or VIP")
        s["membership.type"] = tier
    if status:
        s["membership.status"] = status
    return db.members.find_one_and_update({"member_id": member_id}, {"$set": s}, projection=SAFE,
                                          return_document=ReturnDocument.AFTER)


def assign_trainer(db, member_id, trainer_id):
    t = db.trainers.find_one({"trainer_id": trainer_id}, {"_id": 1})
    if not t:
        raise IntegrityError(f"Trainer {trainer_id} not found")
    return db.members.update_one({"member_id": member_id},
                                 {"$set": {"assigned_trainer_id": t["_id"]}}).modified_count


def delete_member(db, member_id):
    """Blocked if dues are outstanding; otherwise cascades to plans and class rosters.
    Attendance and payment history are kept as the gym's audit trail."""
    oid = _member_oid(db, member_id)
    dues = db.payments.count_documents({"member_id": oid, "status": {"$in": ["pending", "overdue"]}})
    if dues:
        raise IntegrityError(f"{member_id} has {dues} unpaid bill(s); settle before deleting")
    plans = db.workout_plans.delete_many({"member_id": oid}).deleted_count
    rosters = db.classes.update_many({}, {"$pull": {"enrolled_members": {"member_id": oid}}}).modified_count
    db.members.update_many({"referred_by": oid}, {"$set": {"referred_by": None}})
    db.members.delete_one({"_id": oid})
    return {"plans_deleted": plans, "classes_updated": rosters}


# ---------------------------------------------------------------- TRAINERS
def create_trainer(db, name, email, phone, password, specialization, certification, experience_years):
    doc = {"trainer_id": _next_code(db.trainers, "trainer_id", "TRN", 2), "name": name,
           "email": email.lower(), "phone": phone,
           "password_hash": bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode(),
           "role": "trainer", "specialization": specialization, "certification": certification,
           "experience_years": experience_years, "availability": [], "created_at": REFERENCE_DATE}
    db.trainers.insert_one(doc)
    return doc["trainer_id"]


def get_trainer(db, trainer_id):
    return db.trainers.find_one({"trainer_id": trainer_id}, SAFE)


def find_trainers_by_skill(db, skill):
```

```
    return list(db.trainers.find({"specialization": skill}, {"_id": 0, "trainer_id": 1, "name": 1,
                                                             "specialization": 1}))


def add_specialization(db, trainer_id, skill):
    # $addToSet: no duplicates in the embedded array
    return db.trainers.update_one({"trainer_id": trainer_id},
                                  {"$addToSet": {"specialization": skill}}).modified_count


def set_availability(db, trainer_id, day, start_time, end_time):
    db.trainers.update_one({"trainer_id": trainer_id}, {"$pull": {"availability": {"day": day}}})
    return db.trainers.update_one({"trainer_id": trainer_id}, {"$push": {"availability": {
        "day": day, "start_time": start_time, "end_time": end_time}}}).modified_count


def delete_trainer(db, trainer_id):
    t = db.trainers.find_one({"trainer_id": trainer_id}, {"_id": 1})
    if not t:
        raise IntegrityError(f"Trainer {trainer_id} not found")
    members = db.members.count_documents({"assigned_trainer_id": t["_id"]})
    classes = db.classes.count_documents({"trainer_id": t["_id"]})
    if members or classes:
        raise IntegrityError(f"{trainer_id} still has {members} members and {classes} classes; reassign first")
    return db.trainers.delete_one({"_id": t["_id"]}).deleted_count


# ------------------------------------------------------------------ WORKOUT PLANS
def create_workout_plan(db, member_id, goal, duration_weeks, exercises):
    m = db.members.find_one({"member_id": member_id})
    if not m:
        raise IntegrityError(f"Member {member_id} not found")
    t = _require(db, "trainers", m["assigned_trainer_id"], "Trainer")
    # business rule: only one active plan per member
    db.workout_plans.update_many({"member_id": m["_id"], "status": "active"},
                                 {"$set": {"status": "completed", "updated_at": REFERENCE_DATE}})
    doc = {"plan_id": _next_code(db.workout_plans, "plan_id", "WP", 4),
           "member_id": m["_id"], "member_name": m["name"],
           "trainer_id": t["_id"], "trainer_name": t["name"],
           "goal": goal, "duration_weeks": duration_weeks, "status": "active",
           "exercises": exercises, "created_at": REFERENCE_DATE, "updated_at": REFERENCE_DATE}
    db.workout_plans.insert_one(doc)
    return doc["plan_id"]


def get_active_plan(db, member_id):
    return db.workout_plans.find_one({"member_id": _member_oid(db, member_id), "status": "active"},
                                     {"_id": 0, "member_id": 0, "trainer_id": 0})


def add_exercise(db, plan_id, exercise):
    return db.workout_plans.update_one({"plan_id": plan_id},
                                       {"$push": {"exercises": exercise},
                                        "$set": {"updated_at": REFERENCE_DATE}}).modified_count


def progress_exercise(db, plan_id, exercise_name, add_kg):
    """Progressive overload: increase weight of one embedded exercise (positional $ operator)."""
    return db.workout_plans.update_one(
        {"plan_id": plan_id, "exercises.exercise_name": exercise_name},
        {"$inc": {"exercises.$.weight_kg": add_kg}, "$set": {"updated_at": REFERENCE_DATE}}).modified_count
```

```
def remove_exercise(db, plan_id, exercise_name):
    return db.workout_plans.update_one({"plan_id": plan_id},
                                       {"$pull": {"exercises": {"exercise_name": exercise_name}}}).modified_count


def delete_workout_plan(db, plan_id):
    return db.workout_plans.delete_one({"plan_id": plan_id}).deleted_count


# ---------------------------------------------------------------- CLASSES
def create_class(db, class_name, category, trainer_id, capacity, schedule):
    t = db.trainers.find_one({"trainer_id": trainer_id})
    if not t:
        raise IntegrityError(f"Trainer {trainer_id} not found")
    doc = {"class_id": _next_code(db.classes, "class_id", "CLS", 2), "class_name": class_name,
           "category": category, "trainer_id": t["_id"], "trainer_name": t["name"],
           "capacity": capacity, "schedule": schedule, "enrolled_members": []}
    db.classes.insert_one(doc)
    return doc["class_id"]


def get_class(db, class_id):
    return db.classes.find_one({"class_id": class_id}, {"_id": 0, "trainer_id": 0})


def enroll_member(db, class_id, member_id):
    """FR4. Atomic single-document update: the filter itself checks capacity and duplicates,
    so two simultaneous requests can never overbook the class."""
    m = db.members.find_one({"member_id": member_id, "membership.status": "active"})
    if not m:
        raise IntegrityError(f"{member_id} not found or membership not active")
    res = db.classes.update_one(
        {"class_id": class_id,
         "enrolled_members.member_id": {"$ne": m["_id"]},
         "$expr": {"$lt": [{"$size": "$enrolled_members"}, "$capacity"]}},
        {"$push": {"enrolled_members": {"member_id": m["_id"], "member_name": m["name"],
                                        "enrolled_on": REFERENCE_DATE}}})
    if res.modified_count == 0:
        raise IntegrityError(f"Cannot enroll {member_id} in {class_id}: class full or already enrolled")
    return True


def withdraw_member(db, class_id, member_id):
    oid = _member_oid(db, member_id)
    return db.classes.update_one({"class_id": class_id},
                                 {"$pull": {"enrolled_members": {"member_id": oid}}}).modified_count


def member_classes(db, member_id):
    oid = _member_oid(db, member_id)
    return list(db.classes.find({"enrolled_members.member_id": oid},
                                {"_id": 0, "class_id": 1, "class_name": 1, "schedule": 1, "trainer_name": 1}))


def delete_class(db, class_id):
    c = db.classes.find_one({"class_id": class_id}, {"_id": 1})
    if not c:
        raise IntegrityError(f"Class {class_id} not found")
    future = db.class_sessions.delete_many({"class_id": c["_id"], "status": "scheduled"}).deleted_count
    db.classes.delete_one({"_id": c["_id"]})
    return {"future_sessions_removed": future}
```

```
# ---------------------------------------------------------------- SESSIONS & ATTENDANCE
def create_session(db, class_id, date, start_time, end_time):
    c = db.classes.find_one({"class_id": class_id})
    if not c:
        raise IntegrityError(f"Class {class_id} not found")
    doc = {"session_id": _next_code(db.class_sessions, "session_id", "SES", 4), "class_id": c["_id"],
           "class_name": c["class_name"], "trainer_id": c["trainer_id"], "date": date,
           "start_time": start_time, "end_time": end_time, "status": "scheduled"}
    db.class_sessions.insert_one(doc)
    return doc["session_id"]


def cancel_session(db, session_id):
    return db.class_sessions.update_one({"session_id": session_id, "status": "scheduled"},
                                        {"$set": {"status": "cancelled"}}).modified_count


def mark_attendance(db, session_id, member_id, status, check_in_time=None):
    """FR5. Upsert keyed on (session, member): re-marking corrects the record, never duplicates it."""
    s = db.class_sessions.find_one({"session_id": session_id})
    if not s:
        raise IntegrityError(f"Session {session_id} not found")
    m = db.members.find_one({"member_id": member_id})
    if not m:
        raise IntegrityError(f"Member {member_id} not found")
    return db.attendance.update_one(
        {"session_id": s["_id"], "member_id": m["_id"]},
        {"$set": {"status": status, "check_in_time": check_in_time},
         "$setOnInsert": {"class_id": s["class_id"], "member_name": m["name"], "date": s["date"]}},
        upsert=True)


def attendance_history(db, member_id, limit=5):
    oid = _member_oid(db, member_id)
    return list(db.attendance.find({"member_id": oid}, {"_id": 0, "date": 1, "status": 1,
                                                        "check_in_time": 1, "class_id": 1})
                .sort("date", -1).limit(limit))


def delete_attendance(db, session_id, member_id):
    s = db.class_sessions.find_one({"session_id": session_id}, {"_id": 1})
    return db.attendance.delete_one({"session_id": s["_id"], "member_id": _member_oid(db, member_id)}).deleted_count


# ---------------------------------------------------------------- PAYMENTS
def create_invoice(db, member_id, period_start, period_end, due_date):
    m = db.members.find_one({"member_id": member_id})
    if not m:
        raise IntegrityError(f"Member {member_id} not found")
    doc = {"payment_id": _next_code(db.payments, "payment_id", "PAY", 4), "member_id": m["_id"],
           "member_name": m["name"], "amount": TIER_FEES[m["membership"]["type"]], "currency": "INR",
           "payment_mode": None, "status": "pending", "billing_period_start": period_start,
           "billing_period_end": period_end, "payment_date": None, "due_date": due_date}
    db.payments.insert_one(doc)
    return doc["payment_id"]


def record_payment(db, payment_id, mode, paid_on=None):
    res = db.payments.update_one(
        {"payment_id": payment_id, "status": {"$in": ["pending", "overdue"]}},  # paid bills are immutable
        {"$set": {"status": "paid", "payment_mode": mode, "payment_date": paid_on or REFERENCE_DATE}})
    if not res.modified_count:
        return None
```

```
    return db.payments.find_one({"payment_id": payment_id},
                                {"_id": 0, "payment_id": 1, "status": 1, "amount": 1, "payment_mode": 1})


def flag_overdue(db, today=REFERENCE_DATE):
    """FR6. Nightly sweep: any pending bill past its due date becomes overdue (uses ix_due_date)."""
    return db.payments.update_many({"status": "pending", "due_date": {"$lt": today}},
                                   {"$set": {"status": "overdue"}}).modified_count


def member_dues(db, member_id):
    oid = _member_oid(db, member_id)
    return list(db.payments.find({"member_id": oid, "status": {"$ne": "paid"}},
                                 {"_id": 0, "payment_id": 1, "amount": 1, "status": 1, "due_date": 1}))


def delete_payment(db, payment_id):
    # only unpaid invoices raised in error may be deleted; paid records are immutable
    return db.payments.delete_one({"payment_id": payment_id, "status": "pending"}).deleted_count


# ---------------------------------------------------------------------- AUTH (FR8)
def authenticate(db, email, password):
    for coll in ("members", "trainers"):
        u = db[coll].find_one({"email": email.lower()})
        if u and bcrypt.checkpw(password.encode(), u["password_hash"].encode()):
            return {"role": u["role"], "id": u.get("member_id") or u.get("trainer_id"), "name": u["name"]}
    return None
```

Appendix C: Stored Procedures, Functions and Triggers (Full Code)

MongoDB has no SQL stored procedures. Their roles are played by multi-document transactions (a procedure inside BEGIN and COMMIT), server-side aggregation pipelines and views (database functions that return result sets), and change-stream triggers [28], the supported equivalent of CREATE TRIGGER. The complete source of each is given below; nothing is omitted. Table C.1 maps every server-side object to its file.

Table C.1: Location of procedures, functions and triggers

Object	Repository path	In this report
Transactions T1, T2	`gym_db/transactions.py`	C.1
Triggers TR1, TR2	`gym_db/triggers.py`	C.2
Report pipelines A1 to A8, graph queries G1, G2, view `member_dashboard`	`gym_db/aggregations.py`	C.3
Indexes and explain comparison	`gym_db/indexes.py`	C.4
CRUD functions and integrity rules	`gym_db/crud.py`	Appendix B.2
Collections, validators, indexes, view (mongosh)	`mongosh/ddl.js`	Appendix A

C.1 Transactions (gym_db/transactions.py)

```
"""Step 6: multi-document ACID transactions.

Transactions need a replica set (a single-node replica set is enough; see README).
Two business operations touch several collections and must be all-or-nothing:

T1 renew_membership: insert a paid payment AND extend the member's end date.
T2 close_session:    insert attendance rows for a session AND mark it completed.
"""
from datetime import timedelta

from pymongo import ReadPreference
from pymongo.read_concern import ReadConcern
from pymongo.write_concern import WriteConcern

from .config import REFERENCE_DATE, TIER_FEES, get_client, is_mock


class TransactionAborted(Exception):
    pass


def _run(db, fn):
    """Run fn(session) inside a transaction, with automatic retry on transient errors.
    Under mongomock (no transaction support) fn runs without a session."""
    if is_mock():
        return fn(None)
    client = get_client()
    with client.start_session() as s:
        return s.with_transaction(
            fn,
```

```
            read_concern=ReadConcern("snapshot"),
            write_concern=WriteConcern("majority"),
            read_preference=ReadPreference.PRIMARY,
        )


def renew_membership(db, member_id, months, mode="UPI", fail_midway=False):
    """T1. fail_midway=True raises after the payment insert to prove the rollback."""
    def txn(s):
        m = db.members.find_one({"member_id": member_id}, session=s)
        if not m:
            raise TransactionAborted(f"{member_id} not found")
        fee = TIER_FEES[m["membership"]["type"]] * months
        old_end = m["membership"]["end_date"]
        start = max(old_end, REFERENCE_DATE) + timedelta(days=1)
        new_end = start + timedelta(days=30 * months)
        last = db.payments.find_one({}, sort=[("payment_id", -1)], projection={"payment_id": 1}, session=s)
        pid = f"PAY{int(last['payment_id'][3:]) + 1}"
        db.payments.insert_one({
            "payment_id": pid, "member_id": m["_id"], "member_name": m["name"], "amount": fee,
            "currency": "INR", "payment_mode": mode, "status": "paid",
            "billing_period_start": start, "billing_period_end": new_end,
            "payment_date": REFERENCE_DATE, "due_date": REFERENCE_DATE}, session=s)
        if fail_midway:
            raise TransactionAborted("Simulated failure after payment insert")
        db.members.update_one({"_id": m["_id"]},
                              {"$set": {"membership.end_date": new_end, "membership.status": "active"}},
                              session=s)
        return {"payment_id": pid, "amount": fee, "new_end_date": new_end.date().isoformat()}
    return _run(db, txn)


def close_session(db, session_id, marks):
    """T2. marks = {member_id: (status, check_in_time)}. Everything commits together."""
    def txn(s):
        sess = db.class_sessions.find_one({"session_id": session_id, "status": "scheduled"}, session=s)
        if not sess:
            raise TransactionAborted(f"{session_id} is not an open session")
        cls = db.classes.find_one({"_id": sess["class_id"]}, session=s)
        roster = {e["member_id"]: e["member_name"] for e in cls["enrolled_members"]}
        ids = {d["member_id"]: d["_id"] for d in
               db.members.find({"member_id": {"$in": list(marks)}}, {"member_id": 1}, session=s)}
        db.class_sessions.update_one({"_id": sess["_id"]}, {"$set": {"status": "completed"}}, session=s)
        rows = 0
        for mid, (status, t) in marks.items():
            oid = ids.get(mid)
            if oid not in roster:  # fails AFTER earlier writes: the rollback undoes them all
                raise TransactionAborted(f"{mid} is not enrolled in {cls['class_name']}")
            db.attendance.insert_one({"session_id": sess["_id"], "class_id": sess["class_id"],
                                      "member_id": oid, "member_name": roster[oid], "date": sess["date"],
                                      "status": status, "check_in_time": t}, session=s)
            rows += 1
        return {"session_id": session_id, "attendance_rows": rows}
    return _run(db, txn)
```

C.2 Triggers (gym_db/triggers.py)

```
"""Database triggers implemented with MongoDB change streams.

MongoDB has no CREATE TRIGGER statement. The supported equivalent is a change stream:
the server pushes every insert, update, replace and delete on a collection to a listening
client, which reacts to it. Change streams need a replica set, which this project uses.
```

```
Trigger TR1 (audit trail): every change to members and payments is written to the
audit_log collection with the operation, the business key, the fields that changed and
the time. This gives the administrator a tamper-evident history of who paid what and
when a membership changed (NFR3 security, NFR5 consistency).

Trigger TR2 (membership reactivation): when a payment changes to status "paid" for a
member whose membership is "expired", the member is set back to "active".

    python -m gym_db.triggers            # run the triggers until Ctrl+C
"""
import threading
from datetime import datetime, timezone

WATCHED = ["members", "payments"]


def _business_key(db, coll, event):
    doc = event.get("fullDocument") or {}
    key = doc.get("member_id") if coll == "members" else doc.get("payment_id")
    if key is None and coll == "payments" and isinstance(doc.get("member_id"), str):
        key = doc["member_id"]
    return key or str(event["documentKey"]["_id"])


def handle_event(db, event):
    """Apply both triggers to one change event."""
    coll = event["ns"]["coll"]
    op = event["operationType"]
    changed = None
    if op == "update":
        desc = event.get("updateDescription", {})
        changed = sorted(list(desc.get("updatedFields", {}).keys()) + desc.get("removedFields", []))
    # TR1 audit trail
    db.audit_log.insert_one({
        "collection": coll, "operation": op, "key": _business_key(db, coll, event),
        "changed_fields": changed, "at": datetime.now(timezone.utc),
    })
    # TR2 reactivate an expired membership once a bill is paid
    doc = event.get("fullDocument") or {}
    if coll == "payments" and op in ("update", "replace") and doc.get("status") == "paid":
        res = db.members.update_one({"_id": doc["member_id"], "membership.status": "expired"},
                                    {"$set": {"membership.status": "active"}})
        if res.modified_count:
            member = db.members.find_one({"_id": doc["member_id"]}, {"member_id": 1})
            db.audit_log.insert_one({"collection": "members", "operation": "trigger:reactivate",
                                     "key": member["member_id"], "changed_fields": ["membership.status"],
                                     "at": datetime.now(timezone.utc)})


def watch(db, stop_event=None, ready_event=None):
    """Listen to members and payments until stop_event is set."""
    pipeline = [{"$match": {"ns.coll": {"$in": WATCHED}}}]
    with db.watch(pipeline, full_document="updateLookup", max_await_time_ms=200) as stream:
        if ready_event:
            ready_event.set()
        while stop_event is None or not stop_event.is_set():
            event = stream.try_next()
            if event is not None:
                handle_event(db, event)


def start_in_background(db):
```

```
    """Start the triggers on a daemon thread; returns the stop event."""
    stop, ready = threading.Event(), threading.Event()
    threading.Thread(target=watch, args=(db, stop, ready), daemon=True).start()
    ready.wait(5)
    return stop


if __name__ == "__main__":
    from gym_db.config import get_db
    print("Triggers TR1 (audit trail) and TR2 (reactivation) are running. Press Ctrl+C to stop.")
    try:
        watch(get_db())
    except KeyboardInterrupt:
        pass
```

C.3 Aggregation Pipelines, Graph Queries and View (gym_db/aggregations.py)

```
"""Step 5: advanced NoSQL features, aggregation framework (FR7).

Each function returns the pipeline result. Stages used across the set:
$match $group $project $addFields $lookup $unwind $sort $limit $facet $bucket
$graphLookup $cond $size $round $dateToString $filter $sum $avg $push $first
"""
from .config import REFERENCE_DATE, is_mock


# A1. Monthly revenue trend, split by membership tier ($lookup + $group + $dateToString)
def revenue_trend(db):
    return list(db.payments.aggregate([
        {"$match": {"status": "paid"}},
        {"$lookup": {"from": "members", "localField": "member_id", "foreignField": "_id", "as": "m"}},
        {"$unwind": "$m"},
        {"$group": {
            "_id": {"month": {"$dateToString": {"format": "%Y-%m", "date": "$billing_period_start"}},
                    "tier": "$m.membership.type"},
            "revenue": {"$sum": "$amount"}, "payments": {"$sum": 1}}},
        {"$group": {"_id": "$_id.month",
                    "total_revenue": {"$sum": "$revenue"},
                    "payments": {"$sum": "$payments"},
                    "by_tier": {"$push": {"tier": "$_id.tier", "revenue": "$revenue"}}}},
        {"$sort": {"_id": 1}},
        {"$project": {"_id": 0, "month": "$_id", "total_revenue": 1, "payments": 1, "by_tier": 1}},
    ]))


# A2. Class popularity: attendance rate, average turnout and fill rate per class
def class_popularity(db):
    return list(db.attendance.aggregate([
        {"$group": {"_id": "$class_id",
                    "marked": {"$sum": 1},
                    "attended": {"$sum": {"$cond": [{"$in": ["$status", ["present", "late"]]}, 1, 0]}},
                    "sessions": {"$addToSet": "$session_id"}}},
        {"$lookup": {"from": "classes", "localField": "_id", "foreignField": "_id", "as": "c"}},
        {"$unwind": "$c"},
        {"$project": {
            "_id": 0, "class_name": "$c.class_name", "trainer": "$c.trainer_name",
            "enrolled": {"$size": "$c.enrolled_members"}, "capacity": "$c.capacity",
            "sessions_held": {"$size": "$sessions"},
            "avg_turnout": {"$round": [{"$divide": ["$attended", {"$size": "$sessions"}]}, 1]},
            "attendance_rate_pct": {"$round": [{"$multiply": [{"$divide": ["$attended", "$marked"]}, 100]}, 1]},
            "fill_rate_pct": {"$round": [{"$multiply": [
                {"$divide": [{"$size": "$c.enrolled_members"}, "$c.capacity"]}, 100]}, 1]}}},
```

```
        {"$sort": {"avg_turnout": -1, "class_name": 1}},
    ]))


# A3. Members at risk of churning: attendance below 60 percent
def at_risk_members(db, threshold=60):
    return list(db.attendance.aggregate([
        {"$group": {"_id": "$member_id", "name": {"$first": "$member_name"},
                    "total": {"$sum": 1},
                    "attended": {"$sum": {"$cond": [{"$eq": ["$status", "absent"]}, 0, 1]}}}},
        {"$addFields": {"rate_pct": {"$round": [{"$multiply": [{"$divide": ["$attended", "$total"]}, 100]}, 1]}}},
        {"$match": {"rate_pct": {"$lt": threshold}, "total": {"$gte": 4}}},
        {"$lookup": {"from": "members", "localField": "_id", "foreignField": "_id", "as": "m"}},
        {"$unwind": "$m"},
        {"$project": {"_id": 0, "member_id": "$m.member_id", "name": 1, "tier": "$m.membership.type",
                      "sessions_marked": "$total", "attended": 1, "rate_pct": 1}},
        {"$sort": {"rate_pct": 1, "member_id": 1}},
    ]))


# A4. Outstanding dues with days overdue and contact details (FR6)
def outstanding_dues(db, today=REFERENCE_DATE):
    return list(db.payments.aggregate([
        {"$match": {"status": {"$in": ["pending", "overdue"]}}},
        {"$addFields": {"days_overdue": {"$max": [0, {"$floor": {
            "$divide": [{"$subtract": [today, "$due_date"]}, 86400000]}}]}}},
        {"$group": {"_id": "$member_id", "name": {"$first": "$member_name"},
                    "bills": {"$sum": 1}, "amount_due": {"$sum": "$amount"},
                    "max_days_overdue": {"$max": "$days_overdue"},
                    "statuses": {"$addToSet": "$status"}}},
        {"$lookup": {"from": "members", "localField": "_id", "foreignField": "_id", "as": "m"}},
        {"$unwind": "$m"},
        {"$project": {"_id": 0, "member_id": "$m.member_id", "name": 1, "phone": "$m.phone",
                      "tier": "$m.membership.type", "bills": 1, "amount_due": 1, "max_days_overdue": 1,
                      "priority": {"$cond": [{"$gte": ["$max_days_overdue", 15]}, "HIGH",
                                             {"$cond": [{"$gt": ["$max_days_overdue", 0]}, "MEDIUM", "LOW"]}]}}},
        {"$sort": {"max_days_overdue": -1, "amount_due": -1, "member_id": 1}},
    ]))


# A5. Trainer workload: members, active plans, classes (three $lookup joins)
def trainer_workload(db):
    return list(db.trainers.aggregate([
        {"$lookup": {"from": "members", "localField": "_id", "foreignField": "assigned_trainer_id", "as": "mem"}},
        {"$lookup": {"from": "workout_plans", "localField": "_id", "foreignField": "trainer_id", "as": "plans"}},
        {"$lookup": {"from": "classes", "localField": "_id", "foreignField": "trainer_id", "as": "cls"}},
        {"$project": {"_id": 0, "trainer_id": 1, "name": 1, "specialization": 1,
                      "members": {"$size": "$mem"},
                      "active_plans": {"$size": {"$filter": {"input": "$plans", "as": "p",
                                                             "cond": {"$eq": ["$$p.status", "active"]}}}},
                      "classes": {"$size": "$cls"},
                      "class_names": "$cls.class_name"}},
        {"$sort": {"members": -1, "trainer_id": 1}},
    ]))


# A6. Membership dashboard in ONE query: $facet runs three sub-pipelines, $bucket groups ages
def membership_overview(db, today=REFERENCE_DATE):
    res = list(db.members.aggregate([
        {"$addFields": {"age": {"$floor": {"$divide": [{"$subtract": [today, "$dob"]}, 31557600000]}}}},
        {"$facet": {
            "by_tier": [{"$group": {"_id": "$membership.type", "count": {"$sum": 1}}}, {"$sort": {"_id": 1}}],
            "by_status": [{"$group": {"_id": "$membership.status", "count": {"$sum": 1}}}, {"$sort": {"_id": 1}}],
```

```
            "age_groups": [{"$bucket": {"groupBy": "$age", "boundaries": [18, 25, 35, 45, 60],
                                        "default": "60+", "output": {"count": {"$sum": 1}}}}],
            "by_city": [{"$group": {"_id": "$address.city", "count": {"$sum": 1}}}, {"$sort": {"count": -1}}],
        }},
    ]))
    return res[0]


# A7. Most prescribed exercises across all active plans ($unwind on embedded array)
def top_exercises(db, limit=8):
    return list(db.workout_plans.aggregate([
        {"$match": {"status": "active"}},
        {"$unwind": "$exercises"},
        {"$group": {"_id": "$exercises.exercise_name", "plans": {"$sum": 1},
                    "avg_sets": {"$avg": "$exercises.sets"}, "avg_weight_kg": {"$avg": "$exercises.weight_kg"}}},
        {"$project": {"_id": 0, "exercise": "$_id", "plans": 1,
                      "avg_sets": {"$round": ["$avg_sets", 1]}, "avg_weight_kg": {"$round": ["$avg_weight_kg", 1]}}},
        {"$sort": {"plans": -1, "exercise": 1}},
        {"$limit": limit},
    ]))


# A8. Peak check-in hours (string operators on check_in_time)
def peak_hours(db):
    return list(db.attendance.aggregate([
        {"$match": {"status": {"$ne": "absent"}}},
        {"$group": {"_id": {"$substr": ["$check_in_time", 0, 2]}, "check_ins": {"$sum": 1}}},
        {"$project": {"_id": 0, "hour": {"$concat": ["$_id", ":00"]}, "check_ins": 1}},
        {"$sort": {"check_ins": -1, "hour": 1}},
    ]))


# G1. Graph traversal: full referral network of a member ($graphLookup, recursive)
def referral_network(db, member_id):
    res = list(db.members.aggregate([
        {"$match": {"member_id": member_id}},
        {"$graphLookup": {"from": "members", "startWith": "$_id", "connectFromField": "_id",
                          "connectToField": "referred_by", "as": "network",
                          "maxDepth": 5, "depthField": "level"}},
        {"$project": {"_id": 0, "member_id": 1, "name": 1,
                      "network": {"$map": {"input": "$network", "as": "n",
                                           "in": {"member_id": "$$n.member_id", "name": "$$n.name",
                                                  "level": {"$add": ["$$n.level", 1]}}}}}},
    ]))
    if res:
        res[0]["network"].sort(key=lambda n: (n["level"], n["member_id"]))
        res[0]["network_size"] = len(res[0]["network"])
    return res[0] if res else None


# G2. Top referrers: who brought in the largest downstream network
def top_referrers(db, limit=5):
    return list(db.members.aggregate([
        {"$match": {"referred_by": None}},
        {"$graphLookup": {"from": "members", "startWith": "$_id", "connectFromField": "_id",
                          "connectToField": "referred_by", "as": "network", "depthField": "level"}},
        {"$project": {"_id": 0, "member_id": 1, "name": 1, "network_size": {"$size": "$network"},
                      "direct": {"$size": {"$filter": {"input": "$network", "as": "n",
                                                       "cond": {"$eq": ["$$n.level", 0]}}}},
                      "levels": {"$add": [{"$max": "$network.level"}, 1]}}},
        {"$match": {"network_size": {"$gt": 0}}},
        {"$sort": {"network_size": -1, "member_id": 1}},
        {"$limit": limit},
```

```
    ]))


# V1. Read-only view powering the member dashboard (FR3); hides password_hash
MEMBER_DASHBOARD_PIPELINE = [
    {"$lookup": {"from": "trainers", "localField": "assigned_trainer_id", "foreignField": "_id", "as": "t"}},
    {"$lookup": {"from": "payments", "localField": "_id", "foreignField": "member_id", "as": "p"}},
    {"$project": {"_id": 0, "member_id": 1, "name": 1, "tier": "$membership.type",
                  "status": "$membership.status", "trainer": {"$first": "$t.name"},
                  "unpaid_amount": {"$sum": {"$map": {
                      "input": {"$filter": {"input": "$p", "as": "x", "cond": {"$ne": ["$$x.status", "paid"]}}},
                      "as": "u", "in": "$$u.amount"}}}}},
]


def create_views(db):
    if is_mock():
        return False
    db.drop_collection("member_dashboard")
    db.command({"create": "member_dashboard", "viewOn": "members", "pipeline": MEMBER_DASHBOARD_PIPELINE})
    return True


def member_dashboard(db, member_id):
    if is_mock():  # mongomock has no views; run the same pipeline directly
        res = list(db.members.aggregate([{"$match": {"member_id": member_id}}] + MEMBER_DASHBOARD_PIPELINE))
        return res[0] if res else None
    return db.member_dashboard.find_one({"member_id": member_id})
```

C.4 Index Creation and Explain (gym_db/indexes.py)

```
"""Step 3: indexes (NFR1 performance) and an explain() comparison.

Implements every index promised in Review 1 section 6.8, plus a text index for class
search and a partial index that only covers unpaid dues.
"""
from pymongo import ASCENDING, DESCENDING, TEXT

from .config import is_mock

INDEXES = {
    "members": [
        ([("email", ASCENDING)], {"unique": True, "name": "uq_email"}),
        ([("member_id", ASCENDING)], {"unique": True, "name": "uq_member_id"}),
        ([("assigned_trainer_id", ASCENDING)], {"name": "ix_trainer"}),
        ([("membership.type", ASCENDING), ("membership.status", ASCENDING)], {"name": "ix_tier_status"}),
        ([("referred_by", ASCENDING)], {"name": "ix_referred_by"}),
    ],
    "trainers": [
        ([("email", ASCENDING)], {"unique": True, "name": "uq_email"}),
        ([("trainer_id", ASCENDING)], {"unique": True, "name": "uq_trainer_id"}),
    ],
    "workout_plans": [
        ([("member_id", ASCENDING), ("status", ASCENDING)], {"name": "ix_member_status"}),
        ([("trainer_id", ASCENDING)], {"name": "ix_trainer"}),
    ],
    "classes": [
        ([("trainer_id", ASCENDING)], {"name": "ix_trainer"}),
        ([("enrolled_members.member_id", ASCENDING)], {"name": "ix_enrolled_member"}),  # multikey
        ([("class_name", TEXT), ("category", TEXT)], {"name": "tx_class_search"}),
    ],
    "class_sessions": [
```

```
        ([("class_id", ASCENDING), ("date", ASCENDING)], {"name": "ix_class_date"}),
    ],
    "attendance": [
        ([("member_id", ASCENDING), ("date", DESCENDING)], {"name": "ix_member_date"}),
        ([("session_id", ASCENDING), ("member_id", ASCENDING)], {"unique": True, "name": "uq_session_member"}),
        ([("class_id", ASCENDING), ("status", ASCENDING)], {"name": "ix_class_status"}),
    ],
    "payments": [
        ([("member_id", ASCENDING), ("status", ASCENDING)], {"name": "ix_member_status"}),
        ([("due_date", ASCENDING)], {"name": "ix_due_date"}),
        ([("status", ASCENDING), ("due_date", ASCENDING)],
         {"name": "px_unpaid_dues", "partialFilterExpression": {"status": {"$in": ["pending", "overdue"]}}}),
    ],
}


def create_indexes(db):
    created = []
    for coll, specs in INDEXES.items():
        for keys, opts in specs:
            if is_mock() and ("partialFilterExpression" in opts or keys[0][1] == TEXT):
                continue  # mongomock does not support partial or text indexes
            db[coll].create_index(keys, **opts)
            created.append(f"{coll}.{opts['name']}")
    return created


def explain_comparison(db):
    """Run the same query with and without an index and compare the query plans.

    Query: a member's attendance history, newest first (dashboard query).
    """
    if is_mock():
        return None
    member_id = db.members.find_one({"member_id": "MEM1001"})["_id"]
    query = {"member_id": member_id}

    def stats(hint):
        plan = db.attendance.find(query).sort("date", -1).hint(hint).explain()
        es = plan["executionStats"]
        stage = plan["queryPlanner"]["winningPlan"]
        stage = stage.get("queryPlan", stage)  # MongoDB 7+ (slot-based engine) nests the plan
        # walk down to the leaf stage (COLLSCAN or IXSCAN)
        while "inputStage" in stage:
            stage = stage["inputStage"]
        return {"stage": stage["stage"], "docs_examined": es["totalDocsExamined"],
                "keys_examined": es["totalKeysExamined"], "returned": es["nReturned"],
                "time_ms": es["executionTimeMillis"]}

    return {"without_index": stats([("$natural", 1)]), "with_index": stats("ix_member_date")}
```

Appendix D: Complete Query Set with Outputs

The MongoDB and SQL text of all fifteen queries is in Section 6.1 (mongosh version: `mongosh/queries.js`). Below is the complete output of the live run.

D.1 Outputs of the Query Set on MongoDB 8.3.7 (query_output.txt)

```
Connected to MongoDB; database 'gym_db' recreated and seeded; reference date 2026-09-24

================================================================================
Q1  [Selection]  Active VIP members living in Chennai
================================================================================
member_id  name           membership.end_date
---------  -------------  -------------------
MEM1014    Divya Das      2026-12-31
MEM1043    Gautham Nair   2027-05-31
MEM1046    Nandini Menon  2027-02-28
(3 matching documents)

================================================================================
Q2  [Selection]  Trainers who teach Yoga or CrossFit with at least 6 years of experience
================================================================================
trainer_id  name             specialization              experience_years
----------  ---------------  --------------------------  ----------------
TRN03       Deepa Krishnan   Yoga, Pilates               9
TRN08       Vivek Nair       CrossFit, Olympic Lifting   8
TRN07       Kavitha Reddy    Yoga, Mobility              7
TRN01       Priya Menon      Strength Training, CrossFit 6
(4 matching documents)

================================================================================
Q3  [Selection]  Unpaid bills already past their due date (oldest first)
================================================================================
payment_id  member_name     amount  status   due_date
----------  --------------  ------  -------  ----------
PAY7054     Divya Menon     1500    overdue  2026-08-05
PAY7023     Isha Joshi      1500    pending  2026-09-05
PAY7027     Gautham Bhat    2500    pending  2026-09-05
PAY7047     Divya Das       4000    pending  2026-09-05
PAY7061     Aishwarya Das   4000    pending  2026-09-05
PAY7073     Karthik Sharma  2500    pending  2026-09-05
PAY7085     Ananya Das      2500    pending  2026-09-05
PAY7089     Vikram Kumar    1500    pending  2026-09-05
(17 matching documents, first 8 shown)

================================================================================
Q4  [Join (inner)]  Active VIP members with their assigned trainer
================================================================================
member_id  name              trainer        certification
---------  ----------------  -------------  -------------
MEM1001    Harish Kumar      Priya Menon    ACE-CPT
MEM1012    Isha Das          Sanjay Pillai  ISSA-CPT
MEM1014    Divya Das         Rohit Sharma   NASM-CPT
MEM1018    Aishwarya Das     Kavitha Reddy  RYT-200
MEM1028    Lakshmi Krishnan  Rohit Sharma   NASM-CPT
MEM1033    Rahul Sharma      Vivek Nair     CF-L2
MEM1035    Dev Varma         Vivek Nair     CF-L2
MEM1038    Nandini Das       Farhan Ali     ACSM-EP
(8 matching documents)
```

```
================================================================================
Q5  [Join (three tables)]  Attendance history of MEM1001 with class name and session time
================================================================================
date        class_name    start_time  status   check_in_time
----------  ------------  ----------  -------  -------------
2026-09-23  Evening Yoga  19:00       present  18:52
2026-09-19  Evening Yoga  17:00       present  16:51
2026-09-16  Evening Yoga  19:00       present  18:50
2026-09-12  Evening Yoga  17:00       present  16:57
2026-09-09  Evening Yoga  19:00       present  19:00
(5 matching documents)

================================================================================
Q6  [Join (left outer)]  Every trainer with the number of classes they teach, including trainers with none
================================================================================
trainer_id  name             classes  class_names
----------  ---------------  -------  -------------------------
TRN03       Deepa Krishnan   2        Morning Yoga, Pilates Core
TRN02       Rohit Sharma     1        HIIT Express
TRN04       Sanjay Pillai    1        Strength 101
TRN05       Meenakshi Iyer   1        Power Zumba
TRN06       Farhan Ali       1        Spin Cycle
TRN07       Kavitha Reddy    1        Evening Yoga
TRN08       Vivek Nair       1        CrossFit Burn
TRN01       Priya Menon      0
(8 matching documents)

================================================================================
Q7  [Grouping and aggregation]  Money collected by payment mode (COUNT, SUM, AVG)
================================================================================
payment_mode  payments  total   average
------------  --------  ------  -------
UPI           101       237500  2351
card          37        97000   2622
cash          15        37500   2500
(3 matching documents)

================================================================================
Q8  [Grouping with HAVING]  Cities with more than 10 members, split by tier
================================================================================
city       members  basic  premium  vip
---------  -------  -----  -------  ---
Chennai    16       6      6        4
Bengaluru  13       6      5        2
Hosur      12       4      2        6
(3 matching documents)

================================================================================
Q9  [Grouping and aggregation]  Attendance outcome by class category
================================================================================
category  marked  present  late  absent
--------  ------  -------  ----  ------
Yoga      160     103      8     49
Zumba     95      69       8     18
CrossFit  75      51       10    14
Strength  70      46       7     17
HIIT      68      46       8     14
Pilates   60      46       5     9
Cardio    52      36       4     12
(7 matching documents)

================================================================================
```

```
Q10  [Subquery (scalar)]  Members whose attendance rate is below the gym-wide average
============================================================================
name              marked  rate_pct  gym_average_pct
----------------  ------  --------  ---------------
Isha Nair         5       20        77.1
Keerthana Iyer    10      40        77.1
Meera Varma       24      45.8      77.1
Gautham Joshi     14      50        77.1
Isha Joshi        10      50        77.1
Karthik Krishnan  8       50        77.1
Siddharth Varma   24      50        77.1
Nandini Menon     10      60        77.1
(8 matching documents)

============================================================================
Q11  [Subquery (correlated)]  Members with more than one unpaid bill
============================================================================
member_id  name                tier     unpaid_bills  amount_due
---------  ------------------  -------  ------------  ----------
MEM1008    Isha Joshi          Basic    2             3000
MEM1009    Gautham Bhat        Premium  2             5000
MEM1014    Divya Das           VIP      2             8000
MEM1016    Divya Menon         Basic    2             3000
MEM1018    Aishwarya Das       VIP      2             8000
MEM1021    Karthik Sharma      Premium  2             5000
MEM1024    Ananya Das          Premium  2             5000
MEM1025    Vikram Kumar        Basic    2             3000
MEM1038    Nandini Das         VIP      2             8000
MEM1044    Kavya Kumar         Premium  2             5000
MEM1045    Siddharth Pillai    Basic    2             3000
MEM1046    Nandini Menon       VIP      2             8000
MEM1047    Varun Das           Basic    2             3000
MEM1049    Ashwin Krishnan     Premium  2             5000
MEM1051    Karthik Shetty      Premium  2             5000
MEM1052    Isha Nair           Premium  2             5000
MEM1054    Keerthana Krishnan  Premium  2             5000
(17 matching documents)

============================================================================
Q12  [Subquery (EXISTS)]  Classes taught by trainers who coach more than 8 members
============================================================================
class_name    trainer_name    trainer_members
------------  --------------  ---------------
Morning Yoga  Deepa Krishnan  12
Pilates Core  Deepa Krishnan  12
Spin Cycle    Farhan Ali      10
Strength 101  Sanjay Pillai   10
(4 matching documents)

============================================================================
Q13  [Set operation (UNION)]  Members enrolled in Morning Yoga or Evening Yoga (each counted once)
============================================================================
morning_yoga: 13  evening_yoga: 19  union_size: 27
result: Aditya Reddy, Dev Varma, Divya Das, Gautham Joshi, Gautham Subramanian, Harish Kumar, Harish Subramanian,
    Isha Das, Isha Joshi, Isha Nair, Karthik Sharma, Karthik Shetty, Keerthana Iyer, Keerthana Krishnan, Lakshmi Bhat,
    Lakshmi Krishnan, Manoj Shetty, Meera Varma, Nandini Das, Pranav Varma, Rahul Reddy, Shreya Iyer, Siddharth Pillai,
    Siddharth Sharma, Siddharth Varma, Sneha Kumar, Varun Das

============================================================================
Q14  [Set operation (INTERSECT)]  Members enrolled in both CrossFit Burn and Strength 101
============================================================================
intersect_size: 5
result: Dev Varma, Divya Menon, Meera Varma, Rahul Reddy, Siddharth Sharma
```

```
================================================================================
Q15  [Set operation (EXCEPT)]  Active members who are not enrolled in any class
================================================================================
active_members: 50  enrolled_somewhere: 47  except_size: 3
result: Ashwin Krishnan, Rahul Sharma, Swathi Varma
```

D.2 Output of the Index Benchmark (benchmark_output.txt)

```
MongoDB 8.3.7: building gym_bench with 200,000 attendance and 60,000 payment documents ...

query                         plan                      docs_examined  keys_examined  returned  median_ms
B1 member attendance history  COLLSCAN                        200,000              0       105      80.46
B1 member attendance history  ix_member_date                     105            105       105        2.1
B2 absentees of one class     COLLSCAN                        200,000              0       998      84.97
B2 absentees of one class     ix_class_status                     998            998       998       5.53
B3 overdue dues report        COLLSCAN                         60,000              0     2,965      39.95
B3 overdue dues report        px_unpaid_dues (partial)          2,965          2,966     2,965      13.38
B3 overdue dues report        full index                        2,965          2,967     2,965      12.78

Index size: px_unpaid_dues 60.0 KB vs full index 604.0 KB
```

Appendix E: Full-Size ER Diagram and Relational Schema

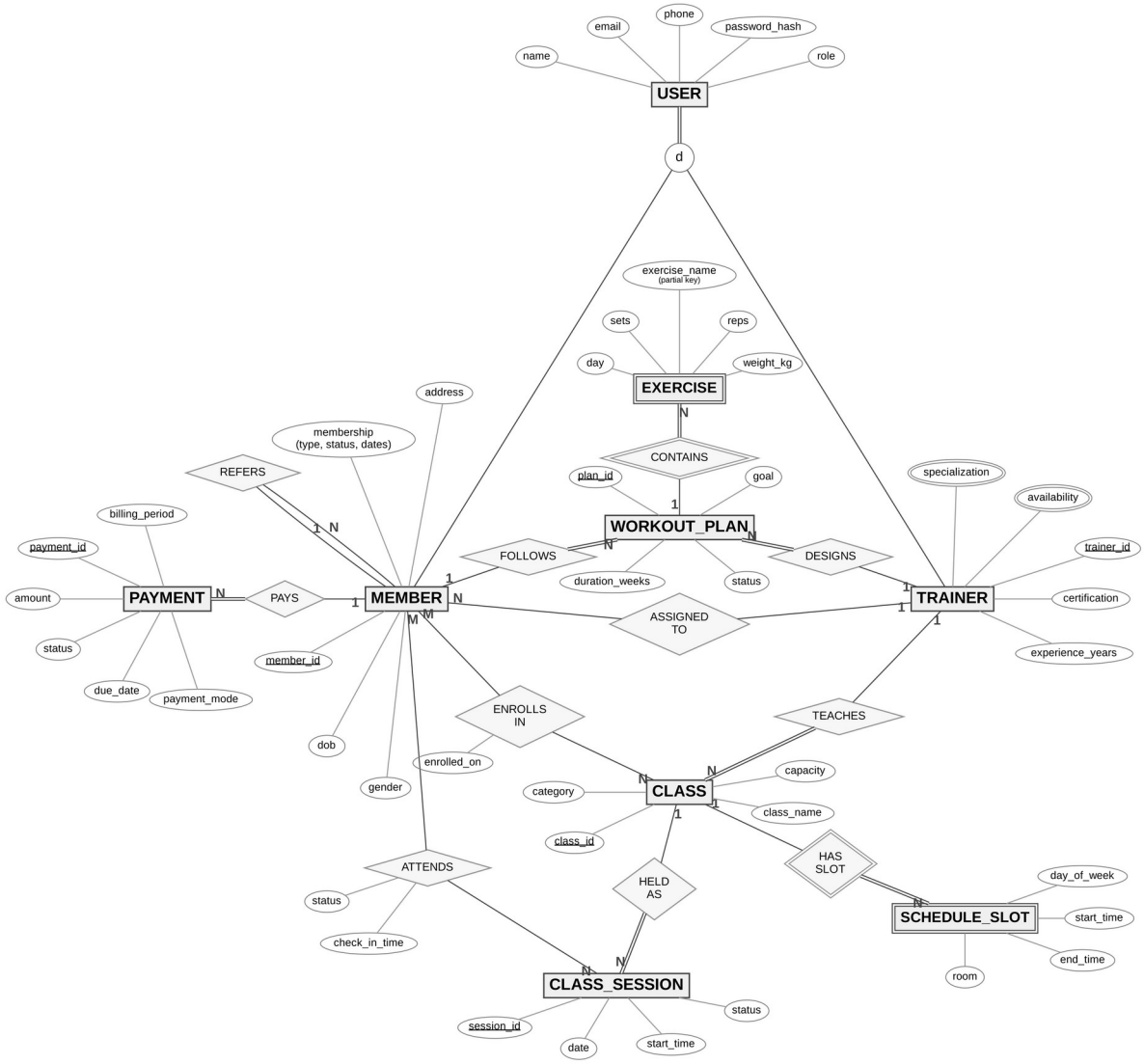


Figure E.1: Complete EER diagram with all attributes

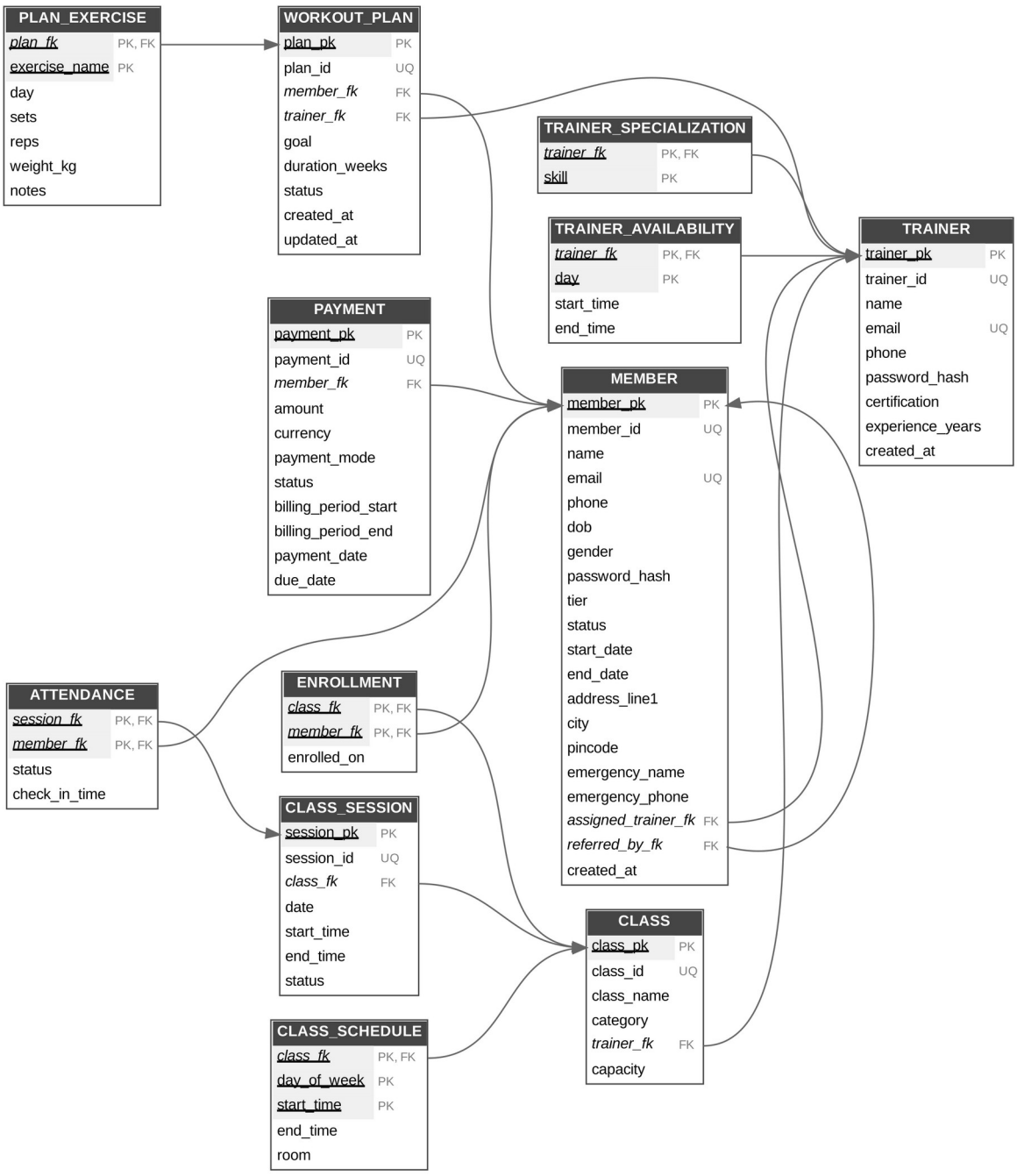


Figure E.2: Full-size relational schema (3NF)

Appendix F: Application Screenshots

Figures F.1 to F.9 are terminal screenshots of the live run on macOS with MongoDB 8.3.11; F.10 to F.12 are from the run on MongoDB 8.3.7.

```
--- Relationship check: one member resolved across all referenced collections
MEM1001 Harish Kumar (VIP) -> trainer TRN01 Priya Menon
  workout plans: 1 | classes: 1 | attendance rows: 5 | payments: 4

--- Validator in action: inserting a member with an invalid tier
REJECTED by validator: 121 Document failed validation
```

Figure F.1: Validator rejecting a member with an invalid tier

```
============================================================================
1. DATABASE IMPLEMENTATION: collections, validators, sample data, indexes
============================================================================
Collections created with $jsonSchema validators:
  members, trainers, workout_plans, classes, class_sessions, attendance, payments

--- Sample data loaded (document counts)
collection      documents
--------------  ---------
trainers        8
members         60
workout_plans   76
classes         8
class_sessions  64
attendance      580
payments        220
TOTAL           1016
```

Figure F.2: Collections created and 1,016 sample documents loaded

```
============================================================================
2. CRUD OPERATIONS
============================================================================

--- CREATE: new trainer, new member (referred by MEM1003), workout plan, class, session, invoice
trainer TRN09 | member MEM1061 | plan WP5077 | class CLS09 | session SES9065 | invoice PAY7221

--- CREATE rejected: duplicate email (unique index uq_email)
DuplicateKeyError: email already registered

--- CREATE rejected: member assigned to a trainer that does not exist (app-level integrity)
IntegrityError: Trainer TRN77 not found

--- READ: member profile MEM1061 (password_hash excluded by projection)
{'_id': ObjectId('6ab528fd6d2803a432e142d5'),
 'member_id': 'MEM1061',
 'name': 'Rhea Thomas',
 'email': 'rhea.thomas@example.com',
 'phone': '9876501234',
 'dob': datetime.datetime(2004, 3, 15, 0, 0),
 'gender': 'Female',
 'role': 'member',
 'membership': {'type': 'Premium',
                'status': 'active',
                'start_date': datetime.datetime(2026, 9, 1, 0, 0),
                'end_date': datetime.datetime(2027, 9, 1, 0, 0)},
 'assigned_trainer_id': ObjectId('6ab528fd6d2803a432e142d4'),
 'referred_by': ObjectId('6ab528f96d2803a432e13ee5'),
 'address': {'line1': '', 'city': 'Vellore', 'pincode': '632014'},
 'emergency_contact': {'name': '', 'phone': ''},
 'created_at': datetime.datetime(2026, 9, 24, 0, 0)}

--- READ: VIP members, page 1 (filter + sort + skip/limit pagination)
11 active VIP members; first 5:
member_id  name              city
---------  ----------------  ---------
MEM1001    Harish Kumar      Bengaluru
MEM1012    Isha Das          Vellore
MEM1014    Divya Das         Chennai
MEM1018    Aishwarya Das     Hosur
MEM1028    Lakshmi Krishnan  Bengaluru

--- READ: trainers who teach Yoga (query inside embedded array)
trainer_id  name
----------  --------------
TRN03       Deepa Krishnan
TRN07       Kavitha Reddy

--- READ: MEM1001 last 5 attendance records (uses index ix_member_date)
date                status   check_in_time
------------------  -------  -------------
2026-09-23 00:00:00  present  18:52
2026-09-19 00:00:00  present  16:51
2026-09-16 00:00:00  present  18:50
2026-09-12 00:00:00  present  16:57
2026-09-09 00:00:00  present  19:00
```

Figure F.3: CRUD demonstration, create and read operations

```
--- READ: MEM1001 last 5 attendance records (uses index ix_member_date)
date                status   check_in_time
------------------  -------  -------------
2026-09-23 00:00:00  present  18:52
2026-09-19 00:00:00  present  16:51
2026-09-16 00:00:00  present  18:50
2026-09-12 00:00:00  present  16:57
2026-09-09 00:00:00  present  19:00

--- UPDATE: enroll into a class of capacity 2 (atomic capacity guard)
3rd enrollment blocked -> Cannot enroll MEM1001 in CLS09: class full or already enrolled
roster: ['Rhea Thomas', 'Swathi Sharma']

--- UPDATE: embedded arrays ($push, positional $inc, $addToSet, $pull)
day  exercise_name  sets  reps  weight_kg
---  -------------  ----  ----  ---------
Mon  Squat          4     10    35
Wed  Bench Press    4     10    25
Fri  Deadlift       3     8     40
specialization: ['Kickboxing', 'HIIT', 'Boxing']

--- UPDATE: tier upgrade Premium -> VIP, and name change propagated to denormalized copies
members: Rhea T. Thomas | tier: VIP
workout_plans.member_name: Rhea T. Thomas
payments.member_name: Rhea T. Thomas
classes.enrolled_members.member_name: ['Rhea T. Thomas']

--- UPDATE: attendance upsert (mark late, then correct to present; still one row)
rows for (SES9065, MEM1061): 1 | status: present

--- UPDATE: payments (record payment, overdue sweep)
record_payment: {'payment_id': 'PAY7221', 'amount': 2500, 'payment_mode': 'UPI', 'status': 'paid'}
bills newly flagged overdue by sweep: 16

--- DELETE: remove exercise, withdraw from class, delete attendance, delete member with cascade
exercises now: ['Squat', 'Bench Press']
withdraw: 1 | delete attendance: 1
delete MEM1016 blocked -> MEM1016 has 2 unpaid bill(s); settle before deleting
delete MEM1061 -> {'plans_deleted': 1, 'classes_updated': 0}
member still exists? False | orphan plans left: 0
delete TRN01 blocked -> TRN01 still has 6 members and 0 classes; reassign first

--- AUTH (FR8): bcrypt login check
correct password: {'role': 'trainer', 'id': 'TRN01', 'name': 'Priya Menon'}
wrong password:   None
```

Figure F.4: CRUD demonstration, update, delete and authentication

```
========================================================================
3a. AGGREGATION PIPELINES (FR7)
========================================================================

--- A1 Monthly revenue trend by tier (paid only)
2026-07  INR  147,000  (60 payments)  [Basic 36,000, Premium 55,000, VIP 56,000]
2026-08  INR  145,500  (59 payments)  [Basic 34,500, Premium 55,000, VIP 56,000]
2026-09  INR   79,500  (34 payments)  [Basic 24,000, Premium 27,500, VIP 28,000]

--- A2 Class popularity
class_name     trainer          enrolled  capacity  fill_rate_pct  sessions_held  avg_turnout  attendance_rate_pct
-------------  ---------------  --------  --------  -------------  -------------  -----------  -------------------
Power Zumba    Meenakshi Iyer   19        25        76.0           5              15.4         81.1
HIIT Express   Rohit Sharma     17        18        94.4           4              13.5         79.4
Evening Yoga   Kavitha Reddy    19        20        95.0           5              12.8         67.4
CrossFit Burn  Vivek Nair       15        15        100.0          5              12.2         81.3
Strength 101   Sanjay Pillai    14        14        100.0          5              10.6         75.7
Pilates Core   Deepa Krishnan   12        12        100.0          5              10.2         85.0
Spin Cycle     Farhan Ali       13        16        81.2           4              10.0         76.9
Morning Yoga   Deepa Krishnan   13        20        65.0           5              9.4          72.3

--- A3 Members at risk (attendance < 60%)
member_id  name              tier     sessions_marked  attended  rate_pct
---------  ----------------  -------  ---------------  --------  --------
MEM1052    Isha Nair         Premium  5                1         20.0
MEM1020    Keerthana Iyer    Premium  10               4         40.0
MEM1030    Meera Varma       Basic    24               11        45.8
MEM1008    Isha Joshi        Basic    10               5         50.0
MEM1019    Siddharth Varma   Basic    24               12        50.0
MEM1053    Karthik Krishnan  Basic    8                4         50.0
MEM1039    Gautham Joshi     Premium  14               7         50.0

--- A4 Outstanding dues (collections worklist)
member_id  name                tier     bills  amount_due  max_days_overdue  priority
---------  ------------------  -------  -----  ----------  ----------------  --------
MEM1016    Divya Menon         Basic    2      3000        50.0              HIGH
MEM1046    Nandini Menon       VIP      2      8000        19.0              HIGH
MEM1038    Nandini Das         VIP      2      8000        19.0              HIGH
MEM1018    Aishwarya Das       VIP      2      8000        19.0              HIGH
MEM1014    Divya Das           VIP      2      8000        19.0              HIGH
MEM1021    Karthik Sharma      Premium  2      5000        19.0              HIGH
MEM1054    Keerthana Krishnan  Premium  2      5000        19.0              HIGH
MEM1044    Kavya Kumar         Premium  2      5000        19.0              HIGH
MEM1052    Isha Nair           Premium  2      5000        19.0              HIGH
MEM1009    Gautham Bhat        Premium  2      5000        19.0              HIGH
MEM1024    Ananya Das          Premium  2      5000        19.0              HIGH
MEM1049    Ashwin Krishnan     Premium  2      5000        19.0              HIGH

--- A5 Trainer workload (3 x $lookup)
trainer_id  name           members  active_plans  classes
----------  -------------  -------  ------------  -------
TRN03       Deepa Krishnan  12       12            2
TRN04       Sanjay Pillai   10       10            1
TRN06       Farhan Ali      10       10            1
TRN08       Vivek Nair      8        8             1
TRN07       Kavitha Reddy   7        7             1
TRN01       Priya Menon     6        6             0
TRN02       Rohit Sharma    5        5             1
TRN05       Meenakshi Iyer  2        2             1
TRN09       Nisha Varghese  0        0             1
```

Figure F.5: Aggregation reports A1 to A5

```
--- A6 Membership overview ($facet + $bucket in one query)
{'by_tier': [{'_id': 'Basic', 'count': 24},
             {'_id': 'Premium', 'count': 22},
             {'_id': 'VIP', 'count': 14}],
 'by_status': [{'_id': 'active', 'count': 50},
               {'_id': 'expired', 'count': 6},
               {'_id': 'frozen', 'count': 4}],
 'age_groups': [{'_id': 18, 'count': 12},
                {'_id': 25, 'count': 15},
                {'_id': 35, 'count': 22},
                {'_id': 45, 'count': 11}],
 'by_city': [{'_id': 'Chennai', 'count': 16},
             {'_id': 'Bengaluru', 'count': 13},
             {'_id': 'Hosur', 'count': 12},
             {'_id': 'Katpadi', 'count': 10},
             {'_id': 'Vellore', 'count': 9}]}

--- A7 Most prescribed exercises ($unwind embedded array)
exercise             plans  avg_sets  avg_weight_kg
-------------------  -----  --------  -------------
Plank                23     4.0       0.0
Rowing Machine       20     3.7       0.0
Squat                19     4.2       67.6
Bench Press          17     4.0       55.9
Deadlift             17     3.9       74.4
Pull Ups             11     4.0       0.0
Treadmill Intervals  11     4.0       0.0
Barbell Row          10     3.6       39.5

--- A8 Peak check-in hours
hour   check_ins
-----  ---------
18:00  115
06:00  98
17:00  82
16:00  51
09:00  39
07:00  24

--- V1 member_dashboard view
{'member_id': 'MEM1005',
 'name': 'Rahul Rao',
 'tier': 'Premium',
 'status': 'expired',
 'trainer': 'Vivek Nair',
 'unpaid_amount': 0}
```

Figure F.6: Aggregation reports A6 to A8 and the member_dashboard view

```
================================================================================
3b. GRAPH TRAVERSAL: referral network with $graphLookup
================================================================================

--- G2 Top referrers (size of downstream network)
member_id  name            direct  network_size  levels
---------  --------------  ------  ------------  ------
MEM1036    Lakshmi Bhat    1       5             3
MEM1016    Divya Menon     2       4             2
MEM1030    Meera Varma     1       4             3
MEM1020    Keerthana Iyer  1       3             3
MEM1051    Karthik Shetty  2       3             2

--- G1 Full referral tree of MEM1036
  L1 MEM1041 Arjun Rao
    L2 MEM1046 Nandini Menon
    L2 MEM1047 Varun Das
      L3 MEM1049 Ashwin Krishnan
      L3 MEM1050 Aishwarya Menon
```

Figure F.7: Referral network with $graphLookup

```
================================================================================
3c. INDEXING
================================================================================

--- Indexes on attendance and payments
  attendance._id_: [('_id', 1)]
  attendance.ix_member_date: [('member_id', 1), ('date', -1)]
  attendance.uq_session_member: [('session_id', 1), ('member_id', 1)] (unique)
  attendance.ix_class_status: [('class_id', 1), ('status', 1)]
  payments._id_: [('_id', 1)]
  payments.ix_member_status: [('member_id', 1), ('status', 1)]
  payments.ix_due_date: [('due_date', 1)]
  payments.px_unpaid_dues: [('status', 1), ('due_date', 1)] (partial)

--- explain(): member attendance history, collection scan vs index scan
plan           stage     docs_examined  keys_examined  returned  time_ms
-------------  --------  -------------  -------------  --------  -------
without_index  COLLSCAN  580            0              5         0
with_index     IXSCAN    5              5              5         0

--- Text index search: classes matching 'yoga'
class_name    category
------------  --------
Evening Yoga  Yoga
Morning Yoga  Yoga
```

Figure F.8: Indexes, explain() comparison and text search

```
================================================================================
3d. MULTI-DOCUMENT ACID TRANSACTIONS
================================================================================

--- T1 renew_membership with a simulated failure -> rollback
aborted: Simulated failure after payment insert
payments count 220 -> 220 (unchanged) | end_date still 2027-03-31

--- T1 renew_membership, 3 months -> commit
{'payment_id': 'PAY7221', 'amount': 4500, 'new_end_date': '2027-06-30'}
end_date 2027-03-31 -> 2027-06-30 | payments 220 -> 221

--- T2 close_session: attendance rows + session status in one transaction
SES9006 Morning Yoga 2026-09-24: {'session_id': 'SES9006', 'attendance_rows': 3}
session status now: completed

--- T2 with a member not on the roster -> whole transaction rolled back
aborted: MEM1001 is not enrolled in Pilates Core
attendance rows 583 -> 583 | SES9046 status: scheduled

Demo complete.
```

Figure F.9: Transactions T1 and T2, commit and rollback

```
(.venv) gym-membership-db % python queries.py   (set operations Q13 to Q15)
================================================================================
Q13  [Set operation (UNION)]  Members enrolled in Morning Yoga or Evening Yoga (each counted once)
================================================================================
morning_yoga: 13  evening_yoga: 19  union_size: 27
result: Aditya Reddy, Dev Varma, Divya Das, Gautham Joshi, Gautham Subramanian, Harish Kumar,
        Harish Subramanian, Isha Das, Isha Joshi, Isha Nair, Karthik Sharma, Karthik Shetty,
        Keerthana Iyer, Keerthana Krishnan, Lakshmi Bhat, Lakshmi Krishnan, Manoj Shetty, Meera Varma,
        Nandini Das, Pranav Varma, Rahul Reddy, Shreya Iyer, Siddharth Pillai, Siddharth Sharma,
        Siddharth Varma, Sneha Kumar, Varun Das

================================================================================
Q14  [Set operation (INTERSECT)]  Members enrolled in both CrossFit Burn and Strength 101
================================================================================
intersect_size: 5
result: Dev Varma, Divya Menon, Meera Varma, Rahul Reddy, Siddharth Sharma

================================================================================
Q15  [Set operation (EXCEPT)]  Active members who are not enrolled in any class
================================================================================
active_members: 50  enrolled_somewhere: 47  except_size: 3
result: Ashwin Krishnan, Rahul Sharma, Swathi Varma
```

Figure F.10: Set operations Q13 to Q15

```
(.venv) gym-membership-db % python benchmark.py
MongoDB 8.3.7: building gym_bench with 200,000 attendance and 60,000 payment documents ...

query                        plan                      docs_examined  keys_examined  returned  median_ms
B1 member attendance history  COLLSCAN                        200,000              0       105      80.46
B1 member attendance history  ix_member_date                      105            105       105       2.1
B2 absentees of one class     COLLSCAN                        200,000              0       998      84.97
B2 absentees of one class     ix_class_status                     998            998       998       5.53
B3 overdue dues report        COLLSCAN                         60,000              0     2,965      39.95
B3 overdue dues report        px_unpaid_dues (partial)          2,965          2,966     2,965      13.38
B3 overdue dues report        full index                        2,965          2,967     2,965      12.78

Index size: px_unpaid_dues 60.0 KB vs full index 604.0 KB
```

Figure F.11: Index benchmark at 200,000 documents

```
(.venv) gym-membership-db % python demo.py --section trig
================================================================================
3e. TRIGGERS: change streams on members and payments
================================================================================

--- TR2: MEM1004 was expired; after paying PAY7223 the trigger set status to active

--- TR1: audit_log written by the trigger
collection  operation          key      changed_fields
----------  -----------------  -------  ---------------------------------
payments    insert             PAY7223
payments    update             PAY7223  payment_date, payment_mode, status
members     trigger:reactivate  MEM1004  membership.status
members     update             MEM1002  phone
members     update             MEM1004  membership.status

Demo complete.
```

Figure F.12: Change-stream triggers TR1 and TR2

Appendix G: User Manual

G.1 Installation

1. Install Python 3.10+, Git, MongoDB Community Server 8 and mongosh (macOS: `brew tap mongodb/brew`, `brew trust mongodb/brew`, `brew install mongodb-community`; Windows: MSI installers from mongodb.com).
2. Enable the replica set: add `replication:` and `replSetName: rs0` to `mongod.conf` (macOS `/opt/homebrew/etc/mongod.conf`; Windows `C:\Program Files\MongoDB\Server\8.0\bin\mongod.cfg`).
3. Restart MongoDB and run `mongosh --eval "rs.initiate()"` once.
4. Clone the repository, create a virtual environment and run `pip install -r requirements.txt` (Section 5.1).

G.2 Running the System

Table G.1: Commands

Command	What it does
`python demo.py`	Rebuilds gym_db and runs every section
`python demo.py --section NAME`	setup, crud, agg, graph, index, txn or trig
`python queries.py [--only Q11]`	The 15 queries (or one)
`python benchmark.py`	Index benchmark (about 10 s)
`python -m pytest -q`	Automated tests (no server needed)
`python demo.py --mock`	Demonstration without a server
`mongosh mongosh/queries.js`	Query set in the shell

G.3 Everyday Tasks

```
from datetime import datetime
from gym_db.config import get_db
from gym_db import crud as C, aggregations as A, transactions as T
db = get_db()
C.create_member(db, "Asha Rao", "asha.rao@example.com", "9876500000", datetime(2002, 5, 1),
                "Female", "Secret@123", tier="Premium", trainer_id="TRN03")
C.enroll_member(db, "CLS01", "MEM1061")                          # book a class
C.mark_attendance(db, "SES9007", "MEM1061", "present", "07:02")  # roll call
T.renew_membership(db, "MEM1061", months=3, mode="UPI")          # renew and pay
C.flag_overdue(db); A.outstanding_dues(db)                       # dues worklist
```

Test logins: `priya.menon@gym.com` / `Trainer@01`; members use `Member@<number>`, for example `Member@1001`.

G.4 Backup and Restore

```
mongodump --db gym_db --out backup/       # backup
mongorestore --drop backup/               # restore
python export_data.py                     # JSON export to data/*.json
```

G.5 Troubleshooting

Table G.2: Common problems

Message	**Fix**
ServerSelectionTimeoutError	Start the MongoDB service
"Transaction numbers are only allowed on a replica set member"	Add `replSetName: rs0` and run `rs.initiate()`
ModuleNotFoundError	Activate `.venv` and install requirements
Untrusted tap (Homebrew)	`brew trust mongodb/brew`
Document failed validation	The value breaks a rule in Table 4.13